

OpenCV

Ch07 Histograms and Matching

OpenCV Tutorials:
Ch3 Image Processing – Part 3

Release 2.4.0

楊曜賓

andy79923@gmail.com

2013.10.28

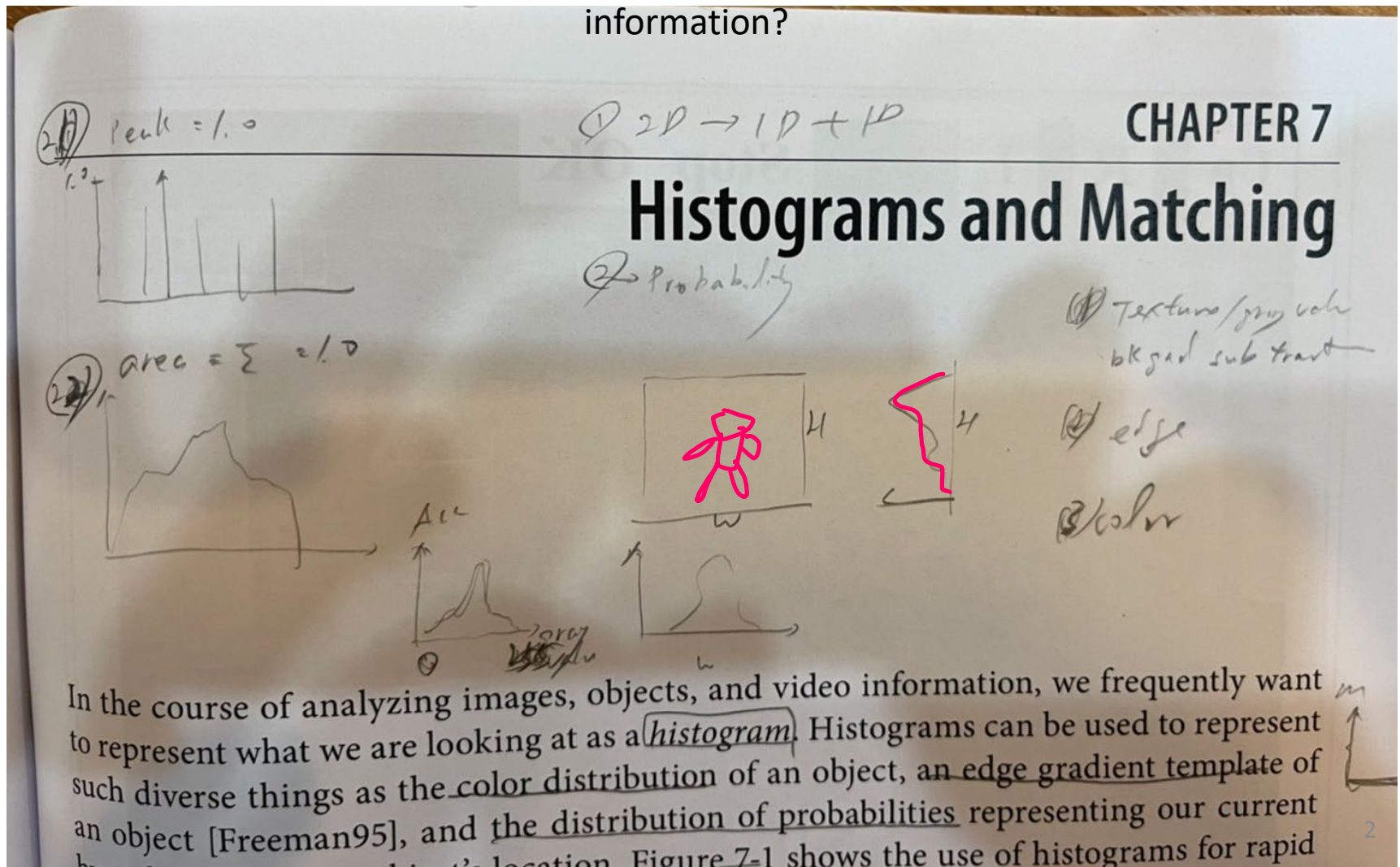
1.1 Histogram Overview

- Histogram

Normalization → Probability

- 2D → 1D + 1D

Keep geometric information or Lose geometric information?



1.1 Histogram Calculation:

Goal

Goal

In this tutorial you will learn how to:

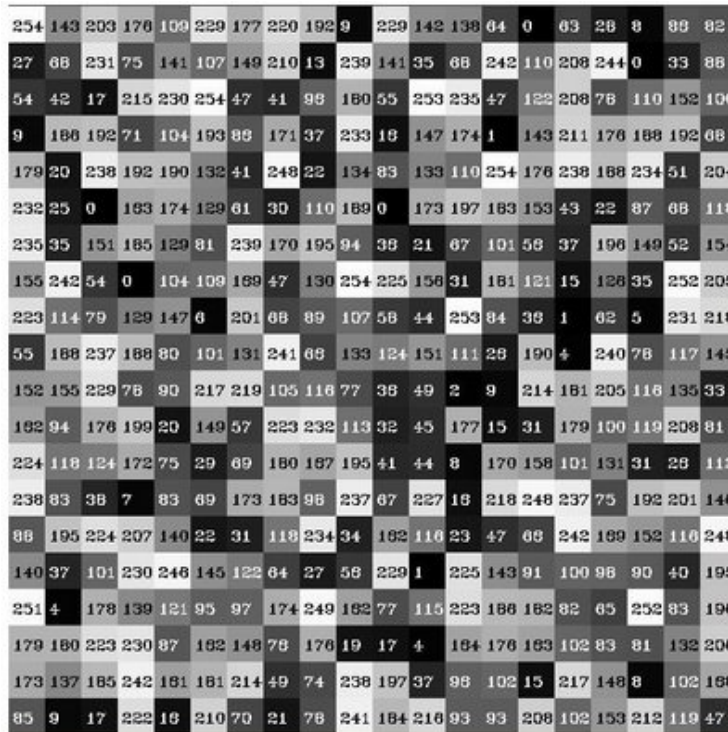
- Use the OpenCV function `split` to divide an image into its correspondent planes.
- To calculate histograms of arrays of images by using the OpenCV function `calcHist`
- To normalize an array by using the function `normalize`

Note: In the last tutorial ([Histogram Equalization](#)) we talked about a particular kind of histogram called *Image histogram*. Now we will considerate it in its more general concept. Read on!

1.1 Histogram Calculation: Definition (1/3)

◆ What are histograms?

1. **Histograms** are **collected counts of data** organized into a set of predefined **bins**.
2. When we say data we are not restricting it to be intensity values. The data collected can be whatever **feature** you find useful to describe your image.



254	143	203	176	100	229	177	220	192	9	229	142	138	64	0	63	26	8	88	82
27	68	231	75	141	107	149	210	13	239	141	35	68	242	110	208	244	0	33	88
54	42	17	215	230	254	47	41	98	180	55	253	235	47	122	208	76	110	152	100
9	186	192	71	104	193	88	171	37	233	18	147	174	1	143	211	176	188	192	68
179	20	238	192	190	132	41	248	22	134	83	133	110	254	176	238	188	234	51	204
232	25	0	183	174	129	61	30	110	189	0	173	197	183	133	43	22	87	68	118
235	35	151	185	129	81	239	170	195	94	38	21	67	101	58	37	198	149	52	154
155	242	54	0	104	109	189	47	130	254	225	156	31	181	121	15	128	35	252	205
223	114	79	129	147	0	201	68	89	107	58	44	253	84	38	1	62	5	231	218
55	188	237	188	80	101	131	241	68	133	124	151	111	28	190	4	240	76	117	145
152	155	229	78	90	217	219	105	116	77	38	49	2	9	214	181	205	116	133	33
182	94	176	199	20	149	57	223	232	113	32	45	177	15	31	179	100	119	208	81
224	118	124	172	75	29	69	180	187	195	41	44	8	170	158	101	131	31	28	112
238	83	38	7	83	69	173	183	98	237	67	227	18	218	248	237	75	192	201	146
88	195	224	207	140	22	31	118	234	34	182	116	23	47	68	242	189	152	116	248
140	37	101	230	248	145	122	64	27	58	229	1	225	143	91	100	98	90	40	195
251	4	178	139	121	95	97	174	249	182	77	115	223	188	182	82	65	252	83	196
179	180	223	230	87	182	148	78	176	19	17	4	184	176	183	102	83	81	132	206
173	137	185	242	181	181	214	49	74	238	197	37	98	102	15	217	148	8	102	188
85	9	17	222	18	210	70	21	78	241	184	216	93	93	208	102	153	212	119	47

A matrix contains information of an image
(i.e. intensity in the range 0-255)

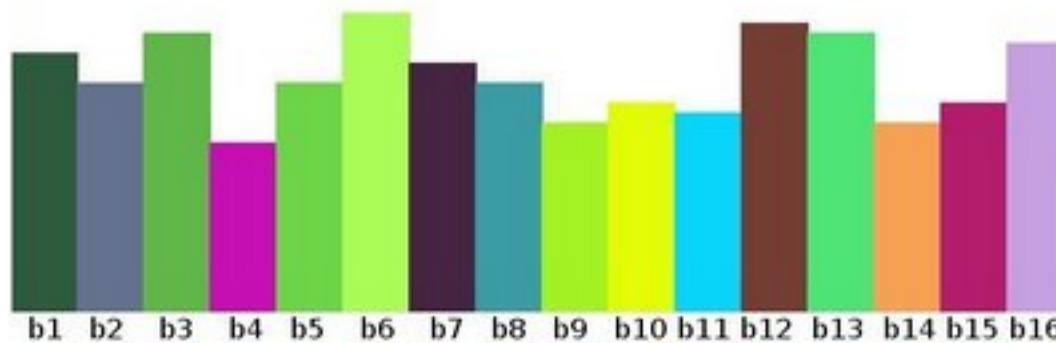
1.1 Histogram Calculation: Definition (2/3)

3. Since the range of information value for image is 256 values, we can segment our range in subparts (called **bins**) like:

$$[0, 255] = [0, 15] \cup [16, 31] \cup \dots \cup [240, 255]$$

$$\text{range} = \text{bin}_1 \cup \text{bin}_2 \cup \dots \cup \text{bin}_{n=16}$$

4. We can keep count of the number of pixels that fall in the range of each **bin_i**



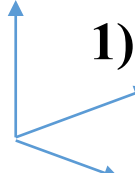
Histogram image (x-axis: bin, pixel value; y-axis: pixel number)

Definition of
X-Axis & Y-Axis

1.1 Histogram Calculation: Definition (3/3)

5. An histogram can keep count not only of **color intensities**, but of whatever **image features** that we want to measure (i.e. gradients, directions, etc).

6. Let's identify some parts of the histogram:

- 
- 1) **dims**: The number of parameters you want to collect data of. In our example, $\text{dims} = 1$ because we are only counting the intensity values of each pixel (in a greyscale image). $\text{Dims}=3$ if RGB channels or HSV
 - 2) **bins**: It is the number of subdivisions in each dim (**x-axis**). In our example, $\text{bins} = 16$
 - 3) **range**: The limits for the values to be measured. In this case: $\text{range} = [0, 255]$

1.1 Histogram Overview

1. Represent diverse things

- 1) Color distribution; Histogram as distribution
RGB 3D distribution → HSV 3D distribution
- 2) Edge gradient template
- 3) Distribution of **probabilities** of object's location

2. Identify interest points

- 1) Assigning each interest point a “tag”
- 2) Edge , colors , corners ...

3. Collect **counts** of the data into **bins**

- 1) Gradient magnitudes , directions, color ...
- 2) Has **fewer dimensions**

4. Represent continuous distributions

- 1) Implicitly averaging the number of point in each **grid cell**



1.1 Histogram Overview

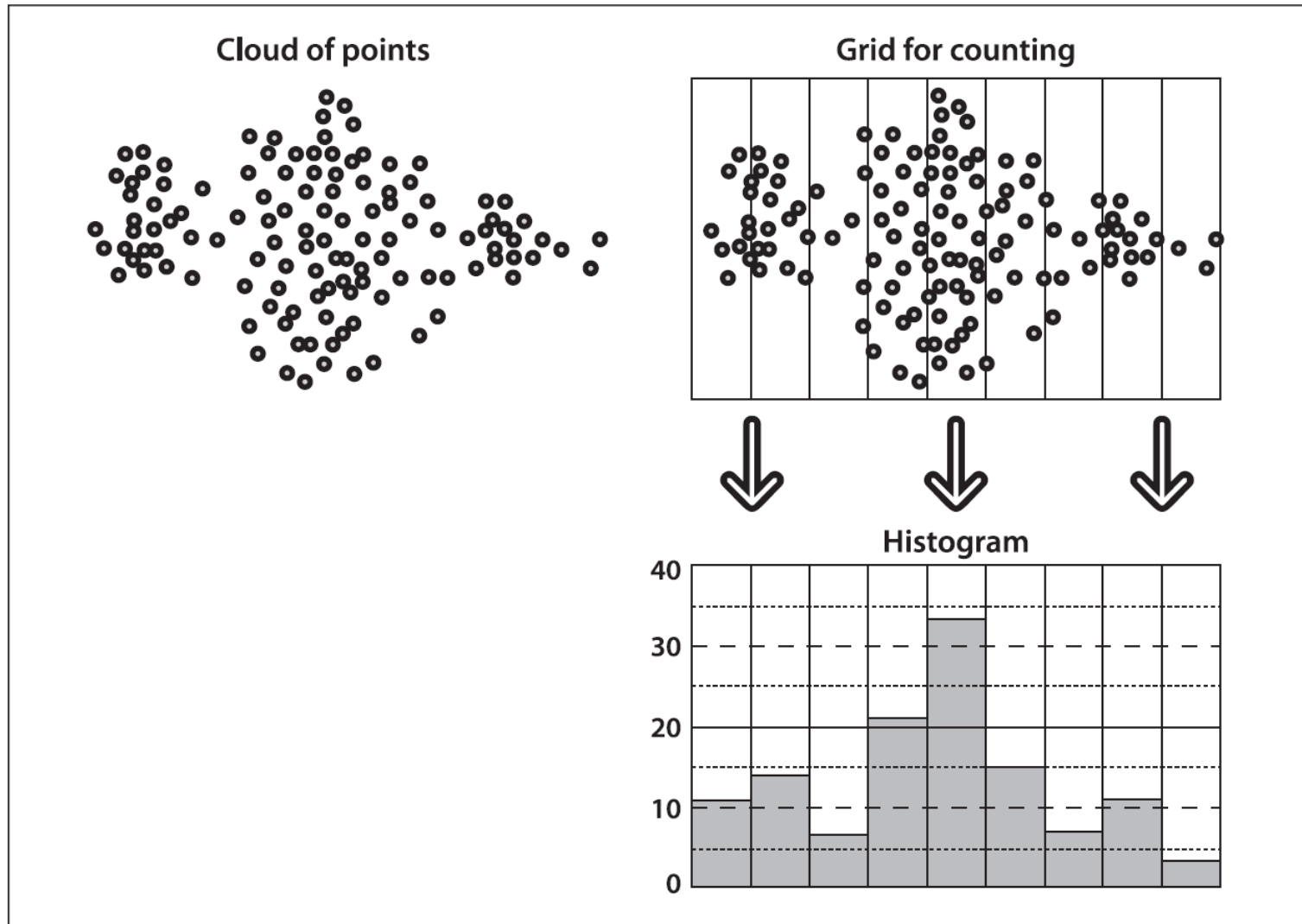


Figure 7-2. Typical histogram example: starting with a cloud of points (upper left), a counting grid is imposed (upper right) that yields a one-dimensional histogram of point counts (lower right)

1.1 Histogram Overview

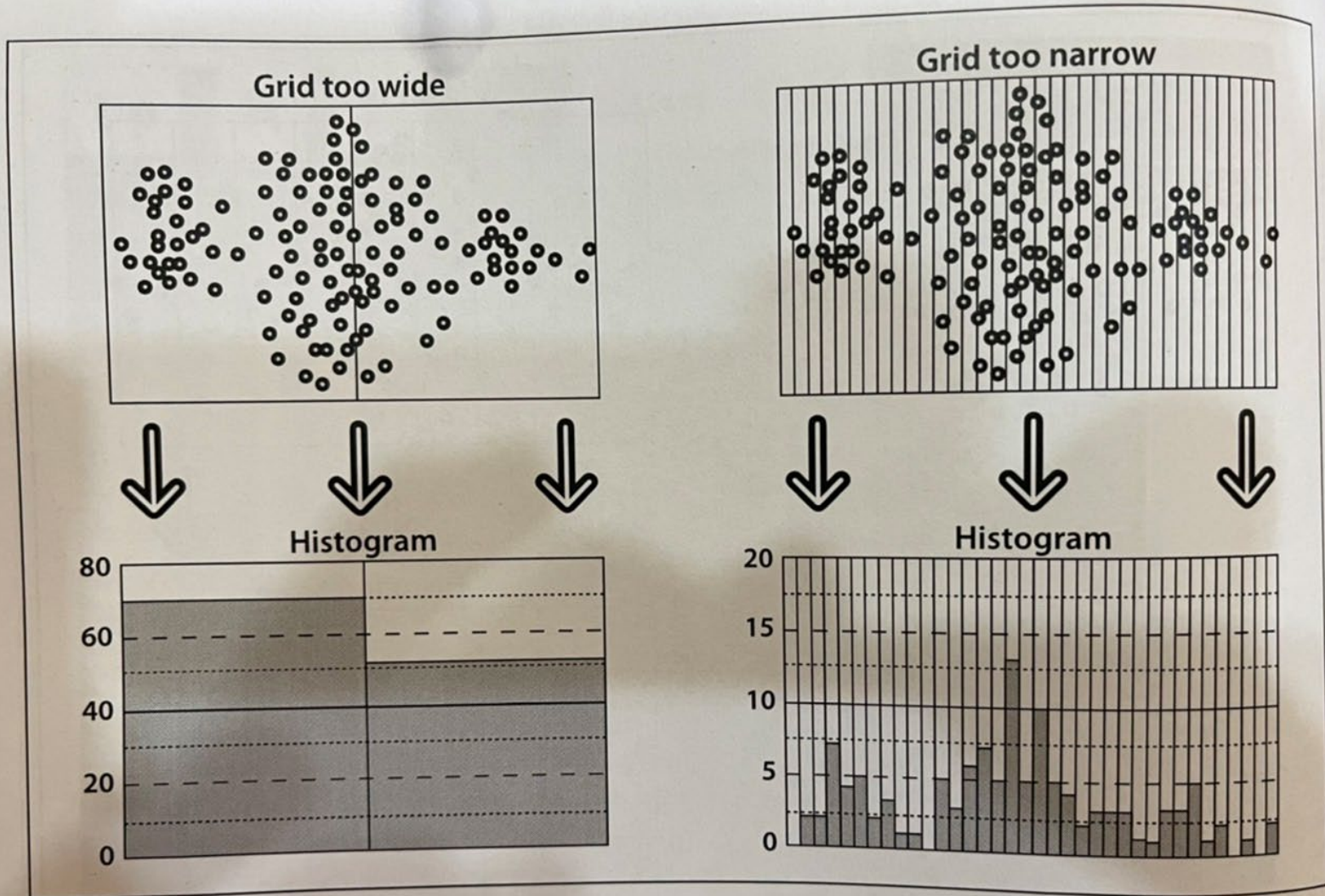


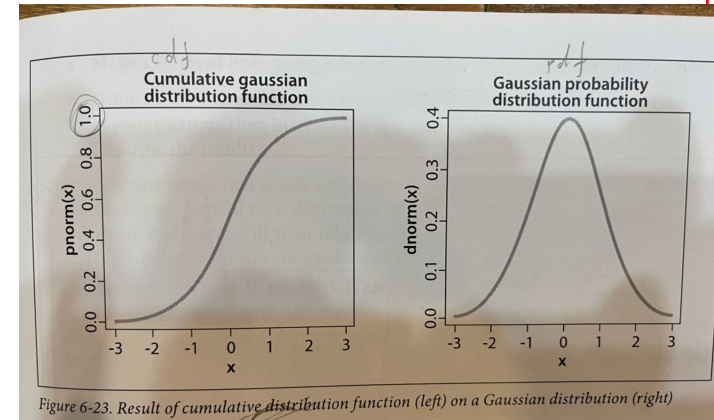
Figure 7-3. A histogram's accuracy depends on its grid size: a grid that is too wide yields too much spatial averaging in the histogram counts (left); a grid that is too small yields "spiky" and singleton results from too little averaging (right)

1.2 Histogram Calculation:

Histogram Normalization ??, jj2

◆ Normalize histogram

1. ??A histogram may also be normalized displaying **relative frequencies**. It then shows **the proportion of cases** that fall into each of several categories.
2. Histograms are used to plot the **density of data**, and often for density estimation:
 - Estimating **the probability density function (pdf)** of the underlying variable.
 - Cdf



3. Normalized to probability $p()=1.0$: 2 methods -

- 1) The total area of a histogram used for **probability density** is always **normalized to 1.0**.
- 2) Peak bin of histogram is **normalized to $p()=1.0$**
4. Application: **Noise reduction/cut** like 5% based on peak bin $p()=1.0=100.0\%$ and then normalize to 100% again.

1.2 Histogram Calculation:

Function Call (1/2) ??

1. Calculates a histogram of a set of arrays

1) calcHist

(1) C++: `void calcHist(const Mat* images, int nimages, const int* channels, InputArray mask, OutputArray hist, int dims, const int* histSize, const float** ranges, bool uniform=true, bool accumulate=false )`

2) Parameters

- (1) `images` – Source arrays. They all should have the same depth, `CV_8U` or `CV_32F`, and the same size. Each of them can have an arbitrary number of channels.
- (2) `nimages` – Number of source images.
- (3) `channels` – List of the dims channels used to compute the histogram. The first array channels are numerated from 0 to `images[0].channels()-1`, the second array channels are counted from `images[0].channels()` to `images[0].channels() + images[1].channels()-1`, and so on.
- (4) `mask` – Optional mask. If the matrix is not empty, it must be an 8-bit array of the same size as `images[i]`. The non-zero mask elements mark the array elements counted in the histogram.
- (5) `hist` – Output histogram.
- (6) `dims` – Output histogram, which is a dense or sparse `dims` -dimensional array.

1.2 Histogram Calculation:

Function Call (2/2) ??

- (7) histSize – Array of histogram sizes in each dimension.
- (8) ?? ranges – x-axis, Array of the dims arrays of the histogram bin boundaries in each dimension. When the histogram is uniform (uniform=true), then for each dimension i it is enough to specify the lower (inclusive) boundary L_0 of the 0-th histogram bin and the upper (exclusive) boundary $U_{\text{histSize}[i]-1}$ for the last histogram bin $\text{histSize}[i]-1$. That is, in case of a uniform histogram each of $\text{ranges}[i]$ is an array of 2 elements. When the histogram is not uniform (uniform=false), then each of $\text{ranges}[i]$ contains $\text{histSize}[i]+1$ elements: $L_0, U_0 = L_1, U_1 = L_2, \dots, U_{\text{histSize}[i]-2} = L_{\text{histSize}[i]-1}, U_{\text{histSize}[i]-1}$. The array elements, that are not between L_0 and $U_{\text{histSize}[i]-1}$, are not counted in the histogram.
- (9) uniform – Flag indicating whether the histogram is uniform or not (see above).
- (10) accumulate – Accumulation flag. If it is set, the histogram is not cleared in the beginning when it is allocated. This feature enables you to compute a single histogram from several sets of arrays, or to update the histogram in time.

1.2 Histogram Calculation: Result 1

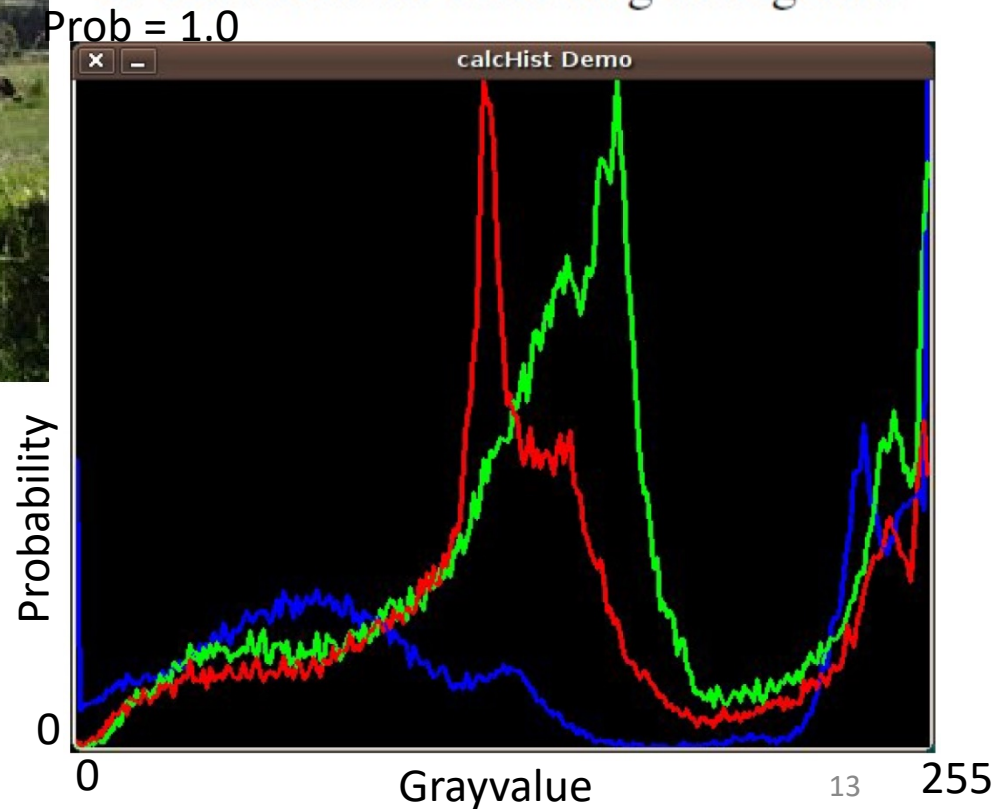
◆ Result 1

1. Using as input argument an image like the shown below:



Distributions of RGB 3 channels, individually

2. Produces the following histogram:



1.2 Histogram Calculation: Result 2

◆ Result 2

1. calcHist

1) Parameters:

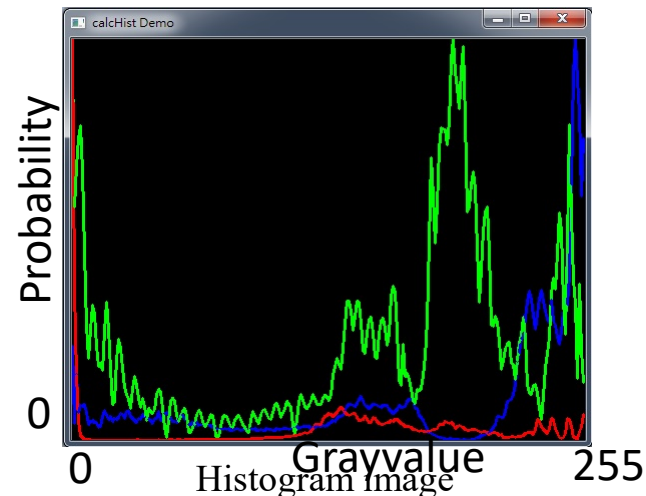
- (1)nimages: 1
- (2)channels: 0
- (3)mask: Mat()
- (4)dims: 1
- (5)histSize: {256,1}
- (6)ranges: {{0,256}}
- (7)uniform: true (default)
- (8)accumulate: false (default)

2. Result:



Input image

Prob = 1.0



1.3 Why from RGB to HSV

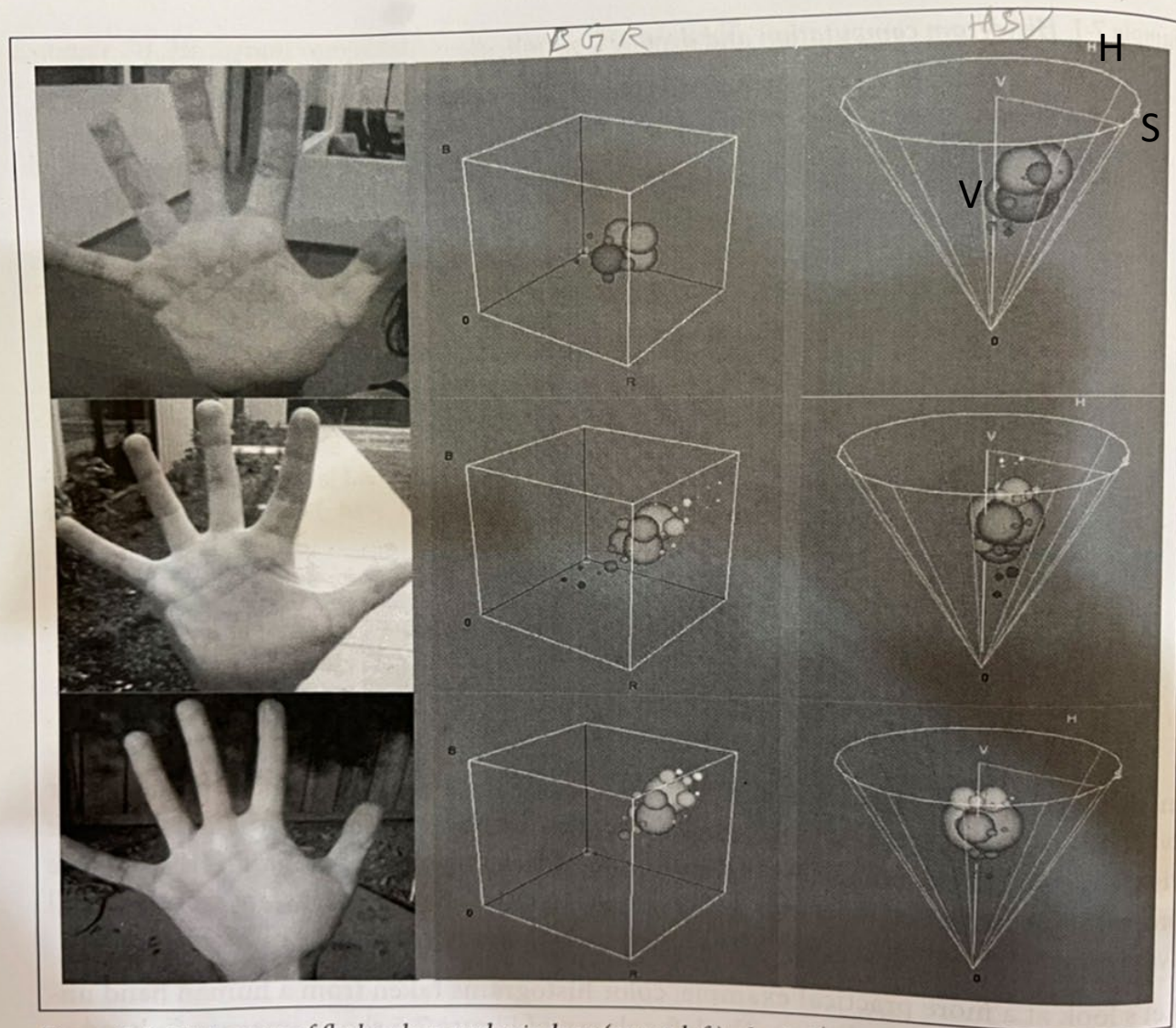


Figure 7-5. Histogram of flesh colors under indoor (upper left), shaded outdoor (middle left), and outdoor (lower left) lighting conditions; the middle and right-hand columns display the associated BGR and HSV histograms, respectively

tools for
portion

Earth

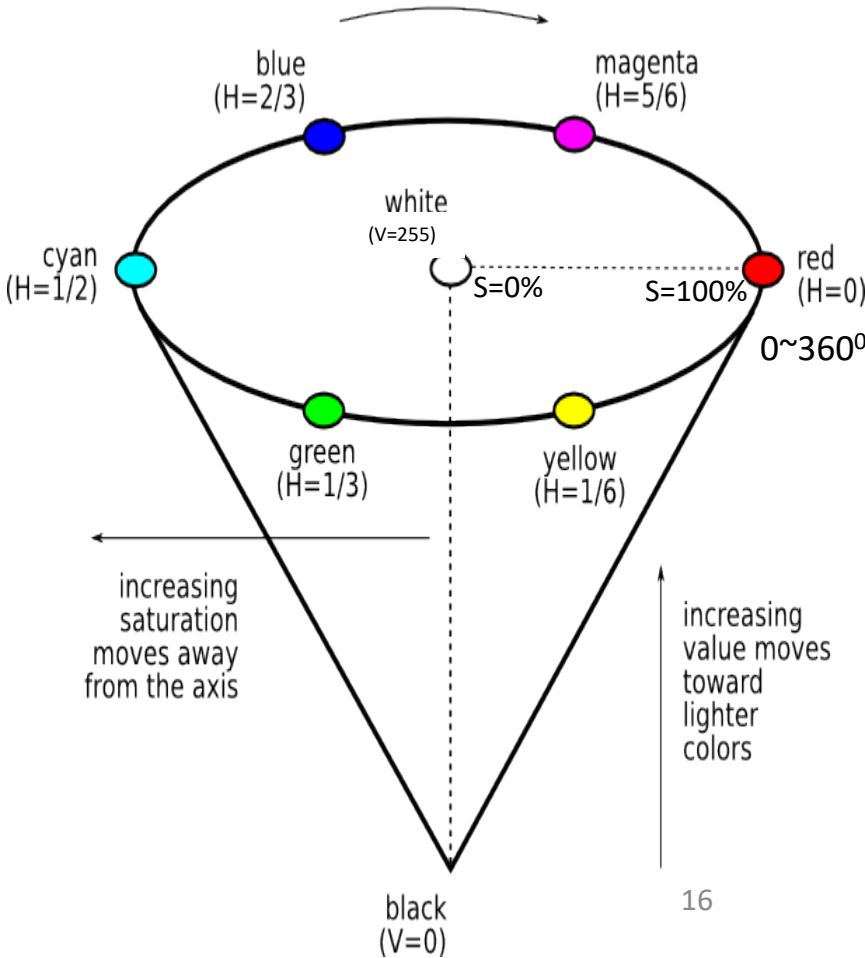
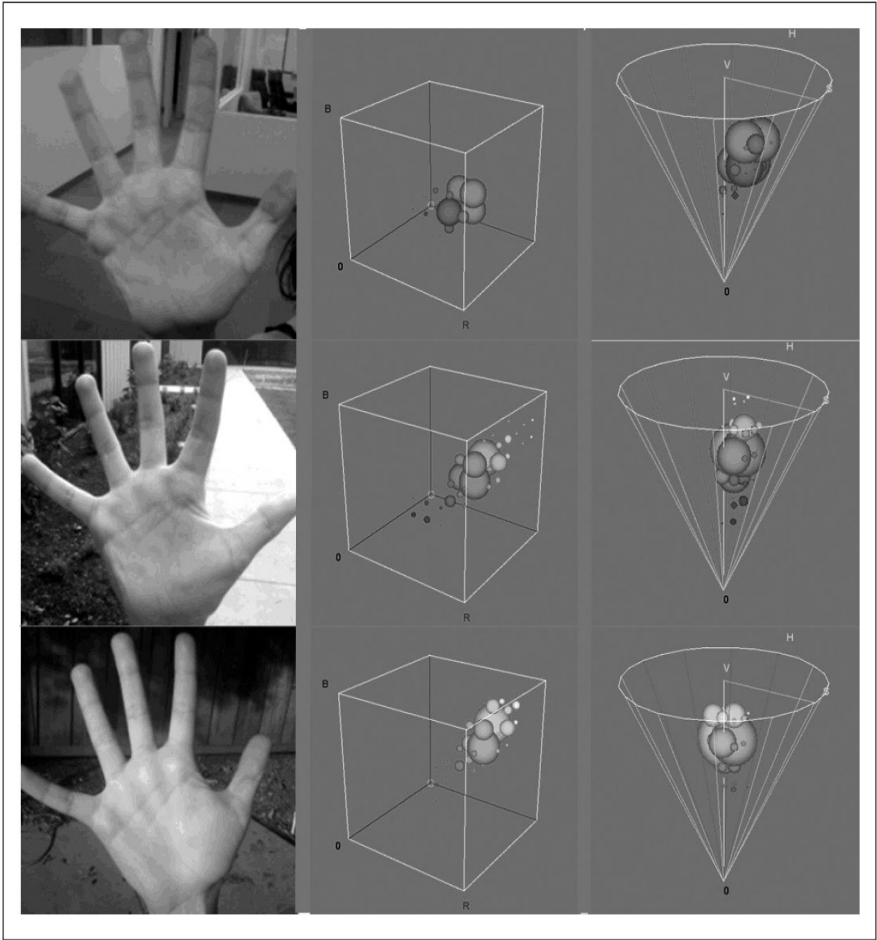
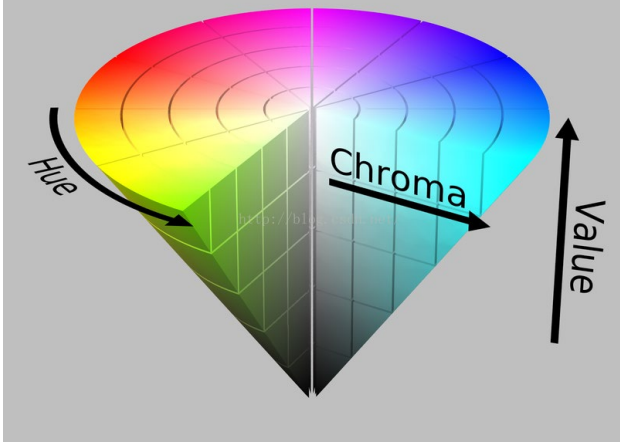
Lighti
tend n
locati
instea
could
has sh
(EMD
take to
histog

Return
sure in
"shove
slot. Fi
tire un
histogr

The EM
ric or t
flowed
rics de
cvCalcE
flo

1.3 HSV

H = Hue 0~360°: Pure color is resemble
S = Saturation 0%~100%: How white the color is
V = Value 0 ~ 255, Brightness : Lightness, how dark the color is



1.4 Histogram Comparison: Goal and Theory

Goal

In this tutorial you will learn how to:

- Use the function `compareHist` to get a numerical parameter that express how well two histograms match with each other.
- Use different metrics to compare histograms

Theory

- To compare two histograms (H_1 and H_2), first we have to choose a *metric* ($d(H_1, H_2)$) to express how well both histograms match.
- OpenCV implements the function `compareHist` to perform a comparison. It also offers 4 different metrics to compute the matching:

1.4 Histogram Comparison: Similarity Measure

◆ Similarity Measure:

1. Histogram (1D+1D): **Fast** but lose feature position information (lose geometric information).

2. Template Matching (2D): **Slow**, accuracy high because containing position information.

1.4 Histogram Comparison: Formula (1/2)

◆ 4 different metrics to compute the matching:

1. Correlation (the higher value the more accurate the match) – Range: [0, 1]

$$d(H_1, H_2) = \frac{\sum_I (H_1(I) - \bar{H}_1)(H_2(I) - \bar{H}_2)}{\sqrt{\sum_I (H_1(I) - \bar{H}_1)^2 \sum_I (H_2(I) - \bar{H}_2)^2}}$$

$$\text{where } \bar{H}_k = \frac{1}{N} \sum_J H_k(J),$$

and N is the total number of histogram bins.

2. Chi-Square (the less value the more accurate the match)

$$d(H_1, H_2) = \sum_I \frac{(H_1(I) - H_2(I))^2}{H_1(I)}$$

1.4 Histogram Comparison:

Formula (2/2) ??2

3. Intersection (the higher value the more accurate the match)

$$d(H_1, H_2) = \sum_I \min(H_1(I), H_2(I))$$

4. Bhattacharyya distance (the less value the more accurate the match) – **Range: [0, 1]**

$$d(H_1, H_2) = \sqrt{1 - \frac{1}{\sqrt{\bar{H}_1 \bar{H}_2 N^2}} \sum_I \sqrt{H_1(I) \cdot H_2(I)}}$$

??

1.4 Histogram Comparison: Result (1/2) ???

The comparison is at
histogram domain, not
image domain

◆ Result

1. We use as input the following images:



Base image



Half base image



Test 1 image



Test 2 image

where the first one is the base (to be compared to the others), the other 2 are the test images. We will also compare the first image with respect to itself and with respect of half the base image.

2. We should expect a perfect match when we compare the base image histogram with itself. Also, compared with the histogram of half the base image, it should present a high match since both are from the same source. For the other two test images, we can observe that they have very different lighting conditions, so the matching should not be very good:

3. Here the numeric results:

<i>Method</i>	Base - Base	Base - Half	Base - Test 1	Base - Test 2
<i>Correlation</i>	1.000000	0.930766	0.182073	0.120447
<i>Chi-square</i>	0.000000	4.940466	21.184536	49.273437
<i>Intersection</i>	24.391548	14.959809	3.889029	5.775088
<i>Bhattacharyya</i>	0.000000	0.222609	0.646576	0.801869

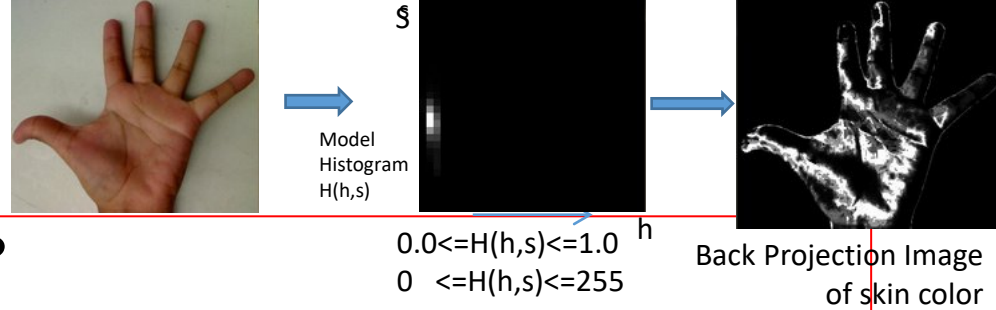
2.1 Back Projection: Goal

Goal

In this tutorial you will learn:

- What is Back Projection and why it is useful
- How to use the OpenCV function `calcBackProject` to calculate Back Projection
- How to mix different channels of an image by using the OpenCV function `mixChannels`

2.1 Back Projection: Definition (1/5) **jj2**



◆ What is Back Projection?

1. Back Projection is a way of recording **how well** the pixels of a given image fit the **distribution (probability)** of pixels in a **histogram model**.
2. For Back Projection, you calculate the histogram model of a feature and then use it to **find this feature (probability)** in an image.
3. Application example: If you have a histogram of flesh color (say, a Hue-Saturation histogram), then you can use it to find flesh color areas, such as skin color, in an image.

2.1 Back Projection:

Definition (2/5) **jj2**

◆ How does it work? (using the skin example)

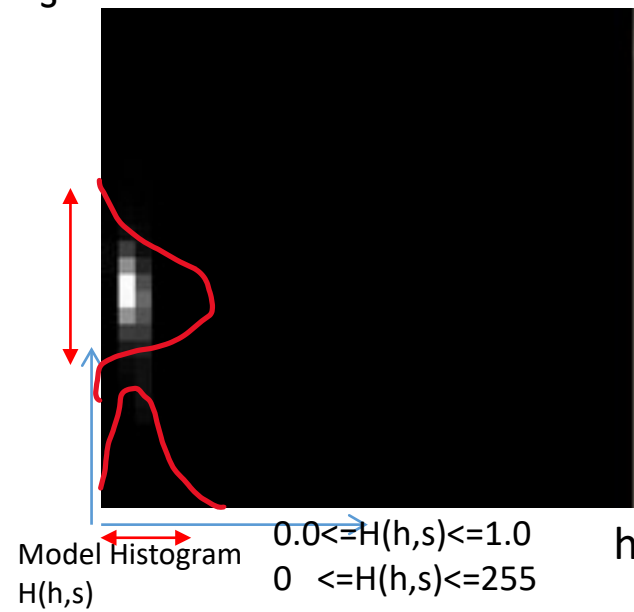
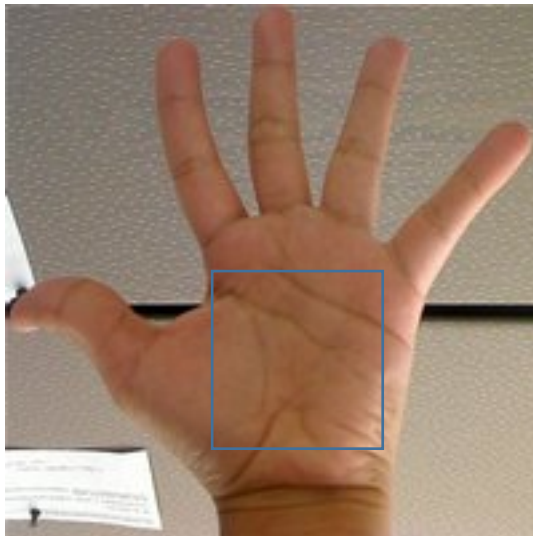
1. Model Histogram: Training database

1) Training data collection –

Let's say you have gotten a **skin histogram** (Hue-Saturation) based on the image below. The histogram besides is going to be our **model histogram** (which we know represents a sample of **skin tonality**). You applied some mask to capture only the histogram of the skin area

2) Histogram normalization –

you may consider to normalize, $0 \leq H(h,s) \leq 255$ (i.e., $0.0 \leq H(h,s) \leq 1.0$), the model histogram first, so the output for the Test Image can be visible for you.

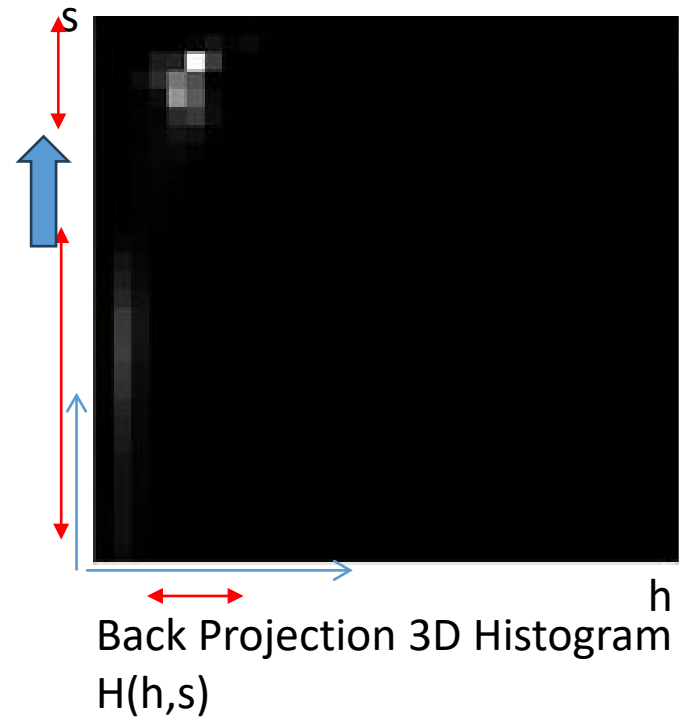


How about
only Hue?

2.1 Back Projection: Definition (3/5) **jj2**

2. Back Projection 3D Histogram: Test image

Now, let's imagine that you get **another hand image (Test Image)** like the one below: (with its respective histogram):



2.1 Back Projection: Definition (4/5) jj2

3. Skin-Color Detection Procedure:

What we want to do is to use our **model histogram** (that we know **represents a skin tonality**) to **detect skin areas** in our Test Image. Here are the steps:

1) 2D bin location ($h_{i,j}, s_{i,j}$):

- In each pixel of our Test Image (i.e. $p(i, j)$), collect the data and find the correspondent 2D bin location for that pixel (i.e. $(h_{i,j}, s_{i,j})$).
 $p(i,j) \Rightarrow 0.0 \leq H(h,s) \leq 1.0$

2) Bin probability value of $H(h,s)$ (Back Projection):

- Lookup the model histogram in the correspondent 2D bin - $(h_{i,j}, s_{i,j})$ - and read the bin value (probability), $H(h,s)$.

3) Back Projection Image:

- Store this bin value in a new image $H(h,s)$ (BackProjection Image).
- Also, you may consider to normalize, $0 \leq H(h,s) \leq 255$ (i.e., $0.0 \leq H(h,s) \leq 1.0$), the model histogram first, so the output for the Test Image can be visible for you.

2.1 Back Projection:

Definition (5/5) jj2

4. Back Projection Image - Skin-Color Detection Image:

What we want to do is to use our **model histogram** (that we know **represents a skin tonality**) to **detect skin areas** in our Test Image. Here are the steps:

- 1) Applying the steps above, we get the following **Back Projection image** for our Test Image:



- 2) In terms of **statistics**, the values stored in Back Projection **represent the probability** that a pixel in Test Image belongs to a skin area, based on the model histogram that we use. For instance in our Test image, the brighter areas are more probable to be skin area (as they actually are), whereas the darker areas have less probability.

- 3) This can apply to **Camshift** algorithm (OpenCV)

2.1 Back Projection:

Function Call (1/2) **jj2**

1. Calculates the back projection of a histogram

1) **calcBackProject**

(1) C++: `void calcBackProject(const Mat* images, int nimages, const int* channels, InputArray hist, OutputArray backProject, const float** ranges, double scale=1, bool uniform=true )`

2) Parameters

- (1) `images` – Source arrays. They all should have the same depth, `CV_8U` or `CV_32F`, and the same size. Each of them can have an arbitrary number of channels.
- (2) `nimages` – Number of source images.
- (3) `channels` – The list of channels used to compute the back projection. The number of channels must match the histogram dimensionality. The first array channels are numerated from 0 to `images[0].channels()-1`, the second array channels are counted from `images[0].channels()` to `images[0].channels() + images[1].channels()-1`, and so on.
- (4) **hist** – Input histogram that can be dense or sparse.

2.1 Back Projection: Function Call (2/2)

- (5) `backProject` – Destination back projection array that is a single-channel array of the same size and depth as `images[0]`.
 - (6) `ranges` – Array of arrays of the histogram bin boundaries in each dimension. See `calcHist()`.
 - (7) `scale` – Optional scale factor for the output back projection.
 - (8) `uniform` – Flag indicating whether the histogram is uniform or not (see above).
- 2) At each location (x, y) the function collects the values from the selected channels in the input images and finds the corresponding histogram bin. But instead of incrementing it, the function reads the bin value, scales it by `scale`, and stores in `backProject(x,y)`.
- 3) In terms of statistics, the function **computes probability** of each element value in respect with the **empirical probability distribution** represented by the histogram.

2.2 Back Projection:

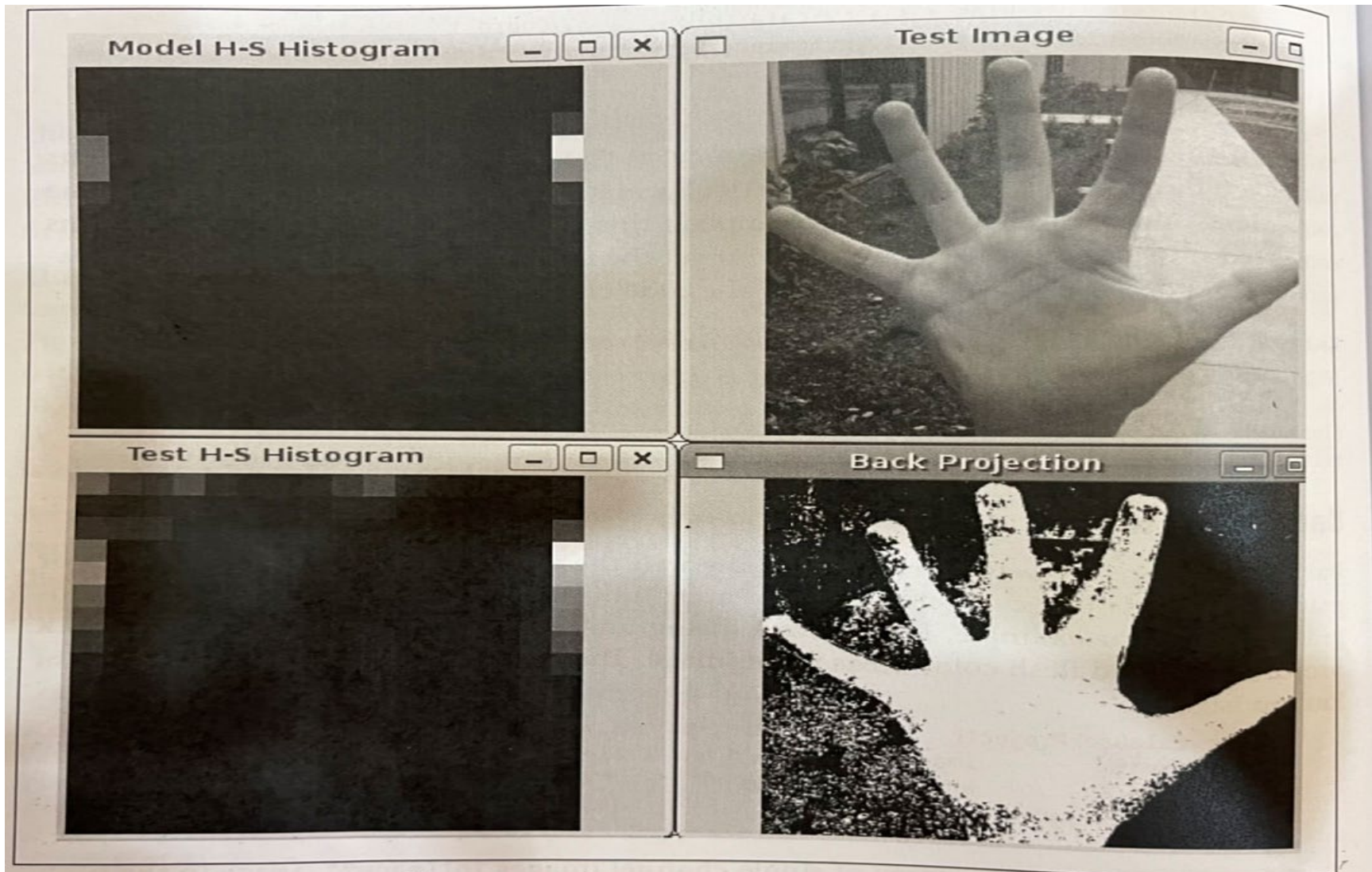


Figure 7-6. Back projection of histogram values onto each pixel based on its color: the HSV flesh-color histogram (upper left) is used to convert the hand image (upper right) into the flesh-color probability image (lower right); the lower left panel is the histogram of the hand image

2.2 Back Projection:

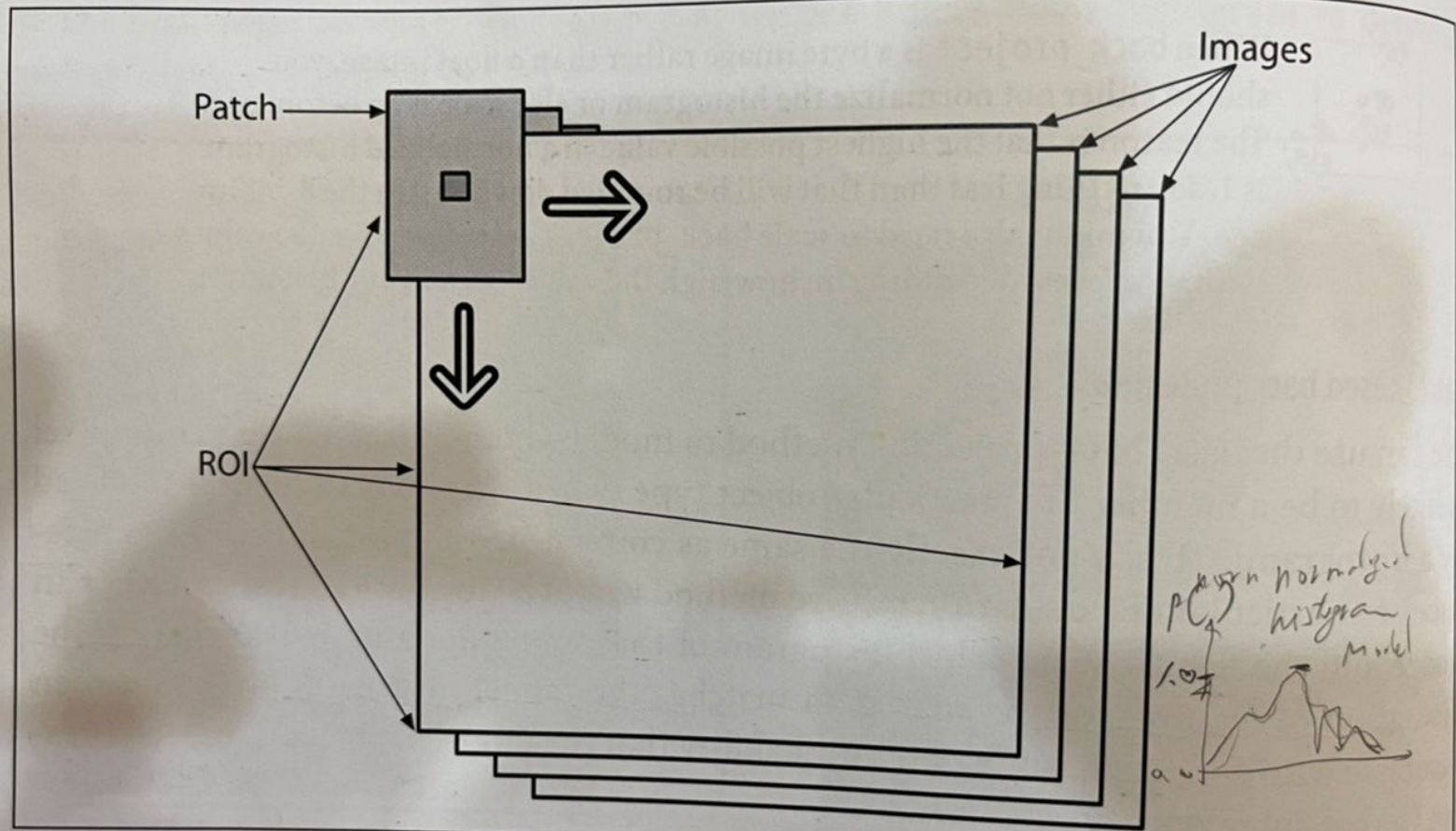


Figure 7-7. Back projection: a sliding patch over the input image planes is used to set the corresponding pixel (at the center of the patch) in the destination image; for normalized histogram models, the resulting image can be interpreted as a probability map indicating the possible presence of the object (this figure is taken from the OpenCV reference manual)

2.2 Back Projection:

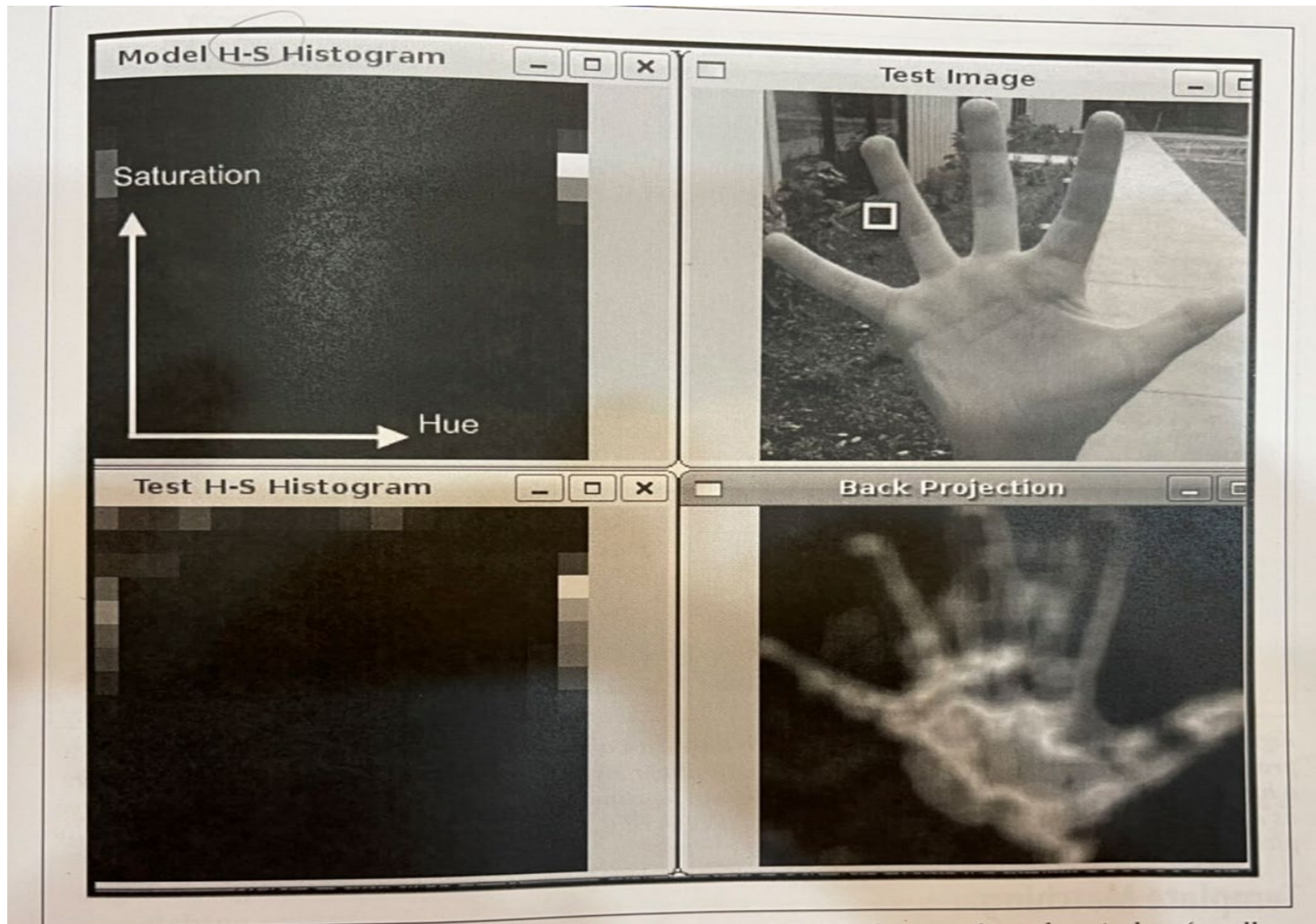


Figure 7-8. Back projection used for histogram object model of flesh tone where the window (small white box in upper right frame) is much smaller than the hand; here, the histogram model was of palm-color distribution and the peak locations tend to be at the center of the hand *his domain*

...tion) * The final parameter factor is the normalization

2.2 Back Projection:

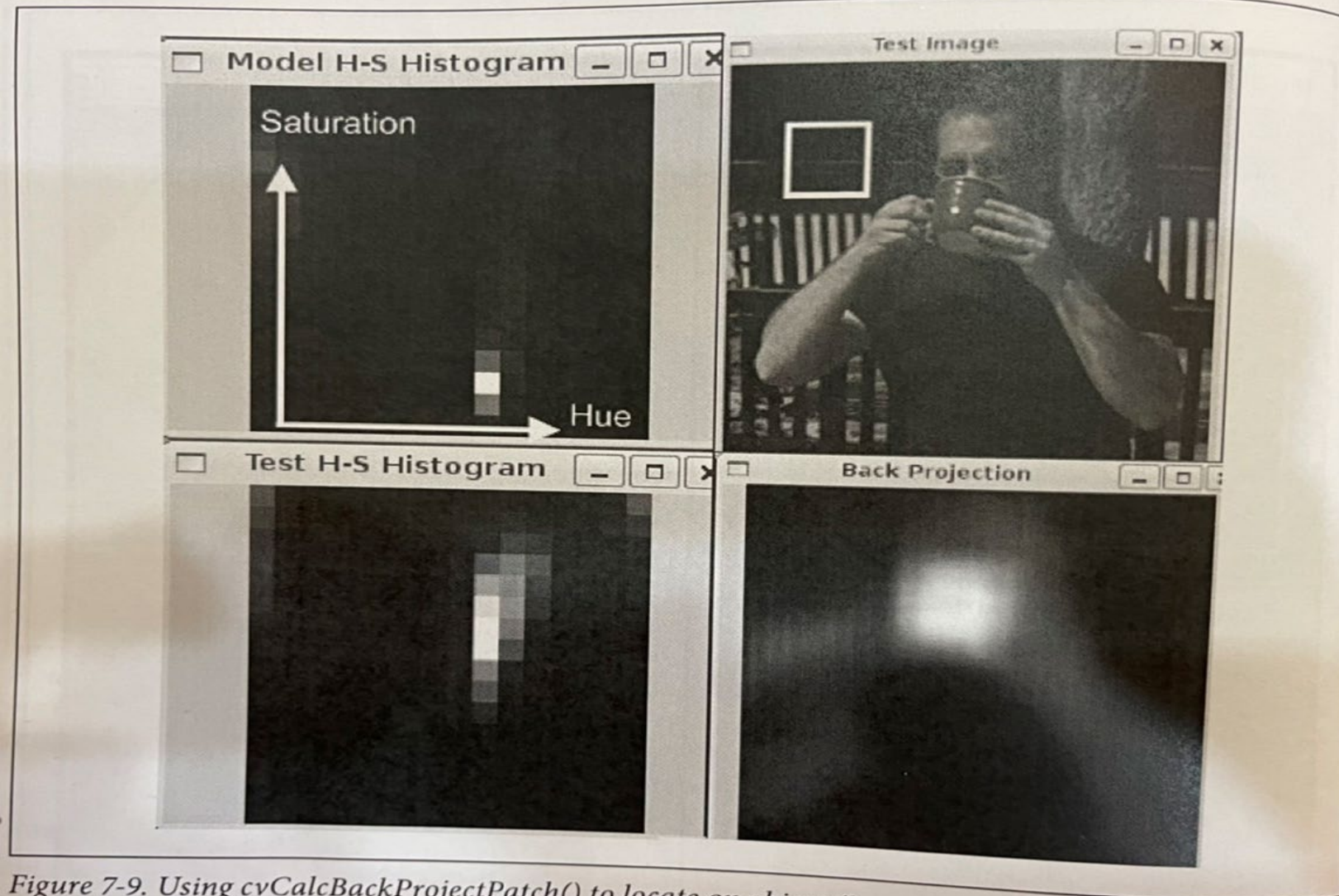
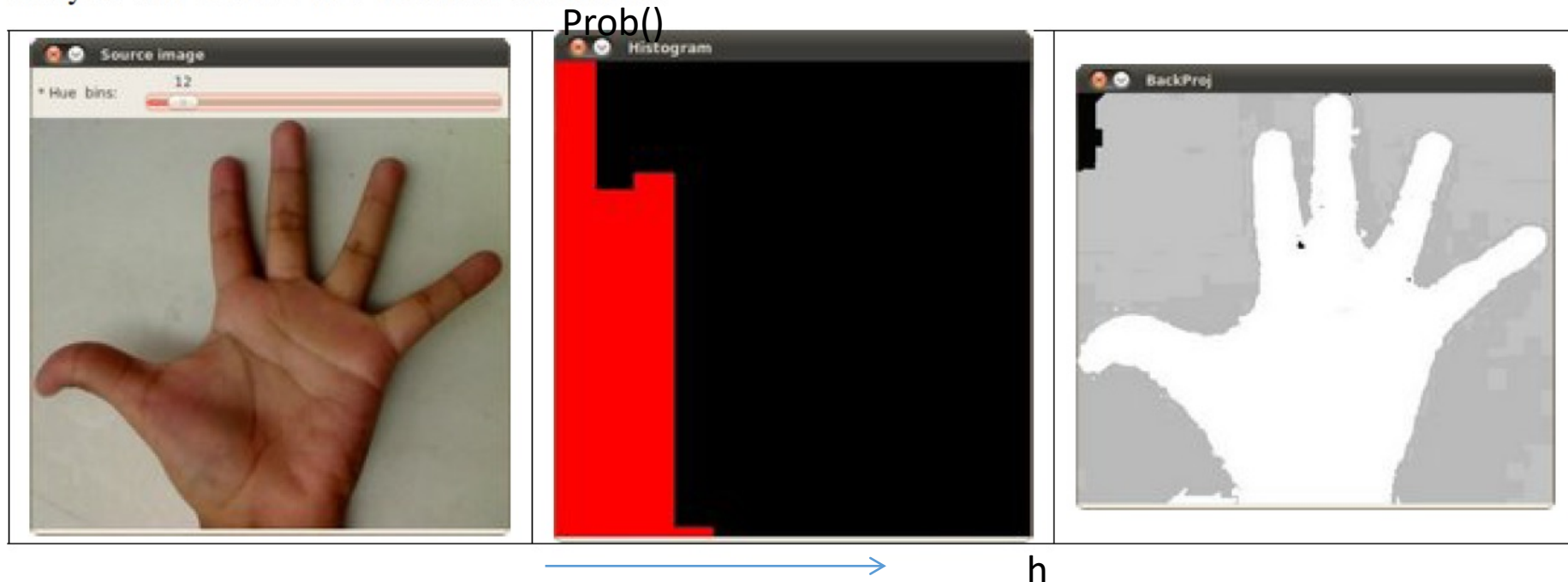


Figure 7-9. Using `cvCalcBackProjectPatch()` to locate an object (here, a coffee cup) whose size approximately matches the patch size (white box in upper right panel): the sought object is modeled by a hue-saturation histogram (upper left), which can be compared with an HS histogram for the image picked out from the scene by virtue of its color

2.2 Back Projection: Result 1 **jj2**

◆ Result 1

1. Here are the output by using a sample image (guess what? Another hand). You can play with the bin values and you will observe how it affects the results:



- Divided into few bins, the result will be smooth/rough
- Divided into more bins, the result will not be smooth/will be in details

2.2 Back Projection: Result 2

◆ Result 2

1. `calcBackProject`

1) Parameter:

(1)`nimages`: 1

(2)`channels`: 0

(3)`ranges`: $\{\{0,180\}\}$

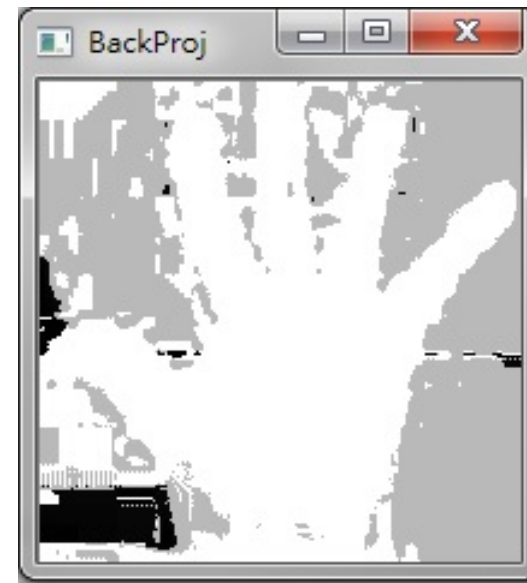
(4)`scale`: 1 (default)

(5)`uniform`: true (default)

2. Result:



Input image



Output image

3.1 Histogram Equalization: Goal and Theory

Goal

In this tutorial you will learn:

- What an image histogram is and why it is useful
- To equalize histograms of images by using the OpenCV function: `equalize_hist` `equalizeHist <>`

Theory

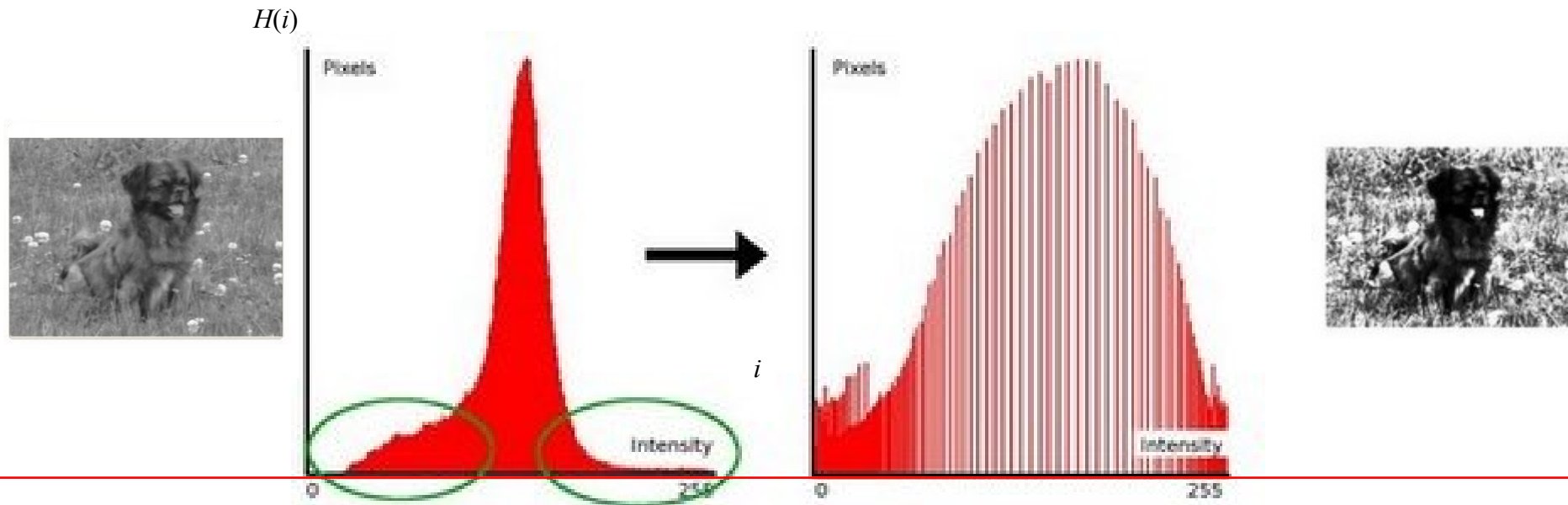
What is an Image Histogram?

- It is a graphical representation of the intensity distribution of an image.
- It quantifies the number of pixels for each intensity value considered.

3.1 Histogram Equalization: Definition

◆ What is Histogram Equalization?

1. It is a method that **improves the contrast** in an image.
2. What Histogram Equalization does is to **stretch out the intensity range** → **Dynamic range (contrast)**.



Histogram Equalization (The green circles indicate the underpopulated intensities.)

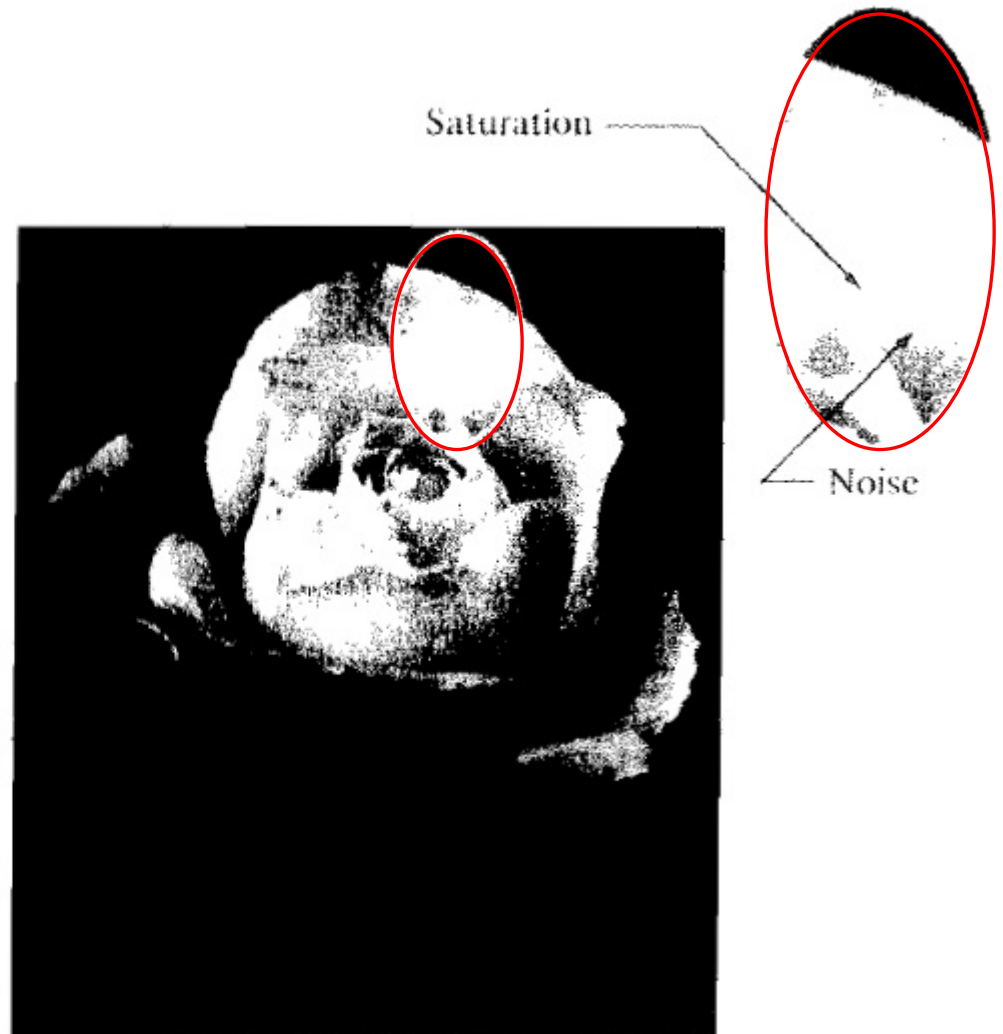
3.1 Histogram Equalization: Dynamic Range (1/2)

◆ Dynamic Range

We assume that the discrete levels are equally spaced and that they are integers in the interval $[0, L - 1]$. Sometimes, the range of values spanned by the gray scale is referred to informally as the dynamic range. This is a term used in different ways in different fields. Here, we define the *dynamic range* of an imaging system to be the ratio of the maximum measurable intensity to the minimum detectable intensity level in the system. As a rule, the upper limit is determined by saturation and the lower limit by noise (see Fig. 2.19). Basically, dynamic range establishes the lowest and highest intensity levels that a system can represent and, consequently, that an image can have. Closely associated with this concept is image contrast, which we define as the difference in intensity between the highest and lowest intensity levels in an image. When an appreciable number of pixels in an image have a high dynamic range, we can expect the image to have high contrast. Conversely, an image with low dynamic range typically has a dull, washed-out gray look.

3.1 Histogram Equalization: Dynamic Range (2/2)

FIGURE 2.19 An image exhibiting saturation and noise. Saturation is the highest value beyond which all intensity levels are clipped (note how the entire saturated area has a high, *constant* intensity level). Noise in this case appears as a grainy texture pattern. Noise, especially in the darker regions of an image (e.g., the stem of the rose) masks the lowest detectable true intensity level.



3.1 Histogram Equalization:

In Figure 6-22 we can see that the variation of the range of values. This is evident from the histogram of its intensity values on the right. Because we are dealing with an 8-bit image, its intensity values can range from 0 to 255, but the histogram shows that the actual intensity values are all clustered near the middle of the available range. Histogram equalization is a method for stretching this range out.

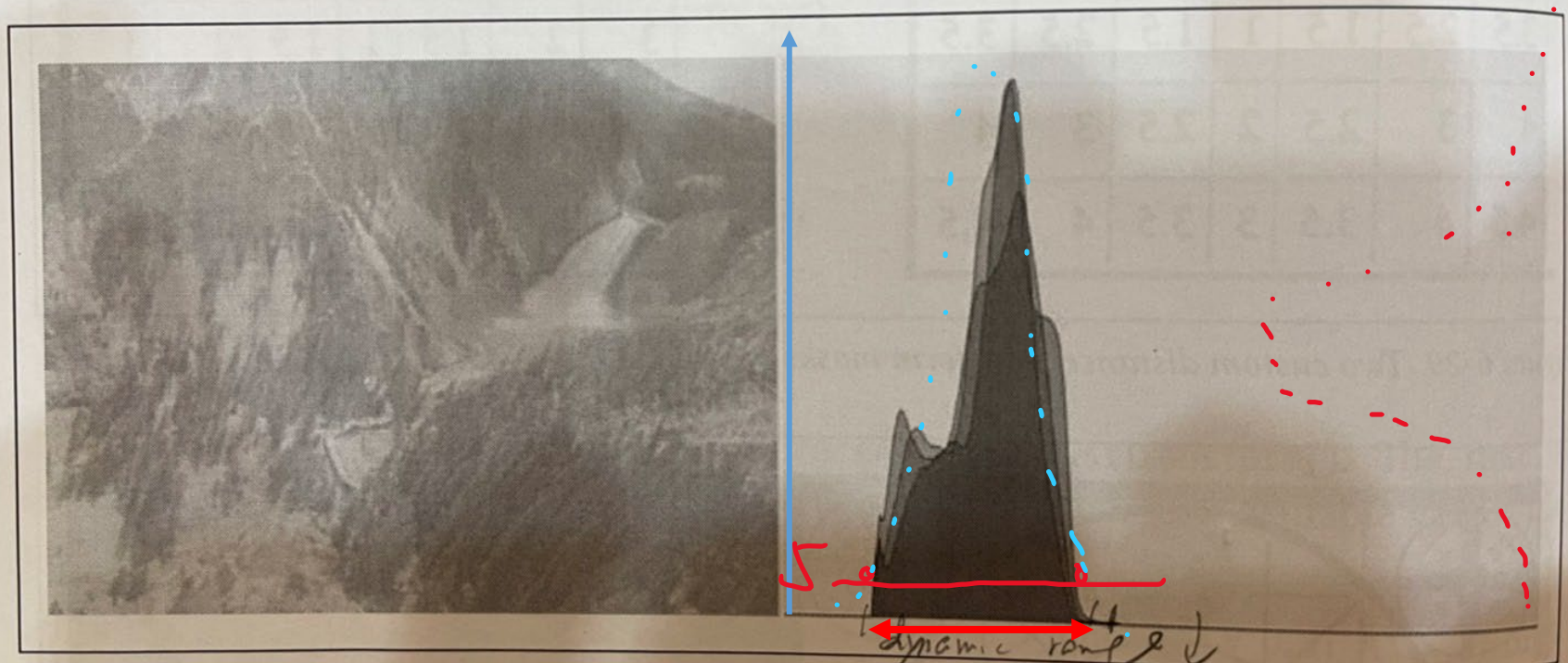


Figure 6-22. The image on the left has poor contrast, as is confirmed by the histogram of its intensity values on the right

3.1 Histogram Equalization:

CDF here, Mapping curve as Gamma

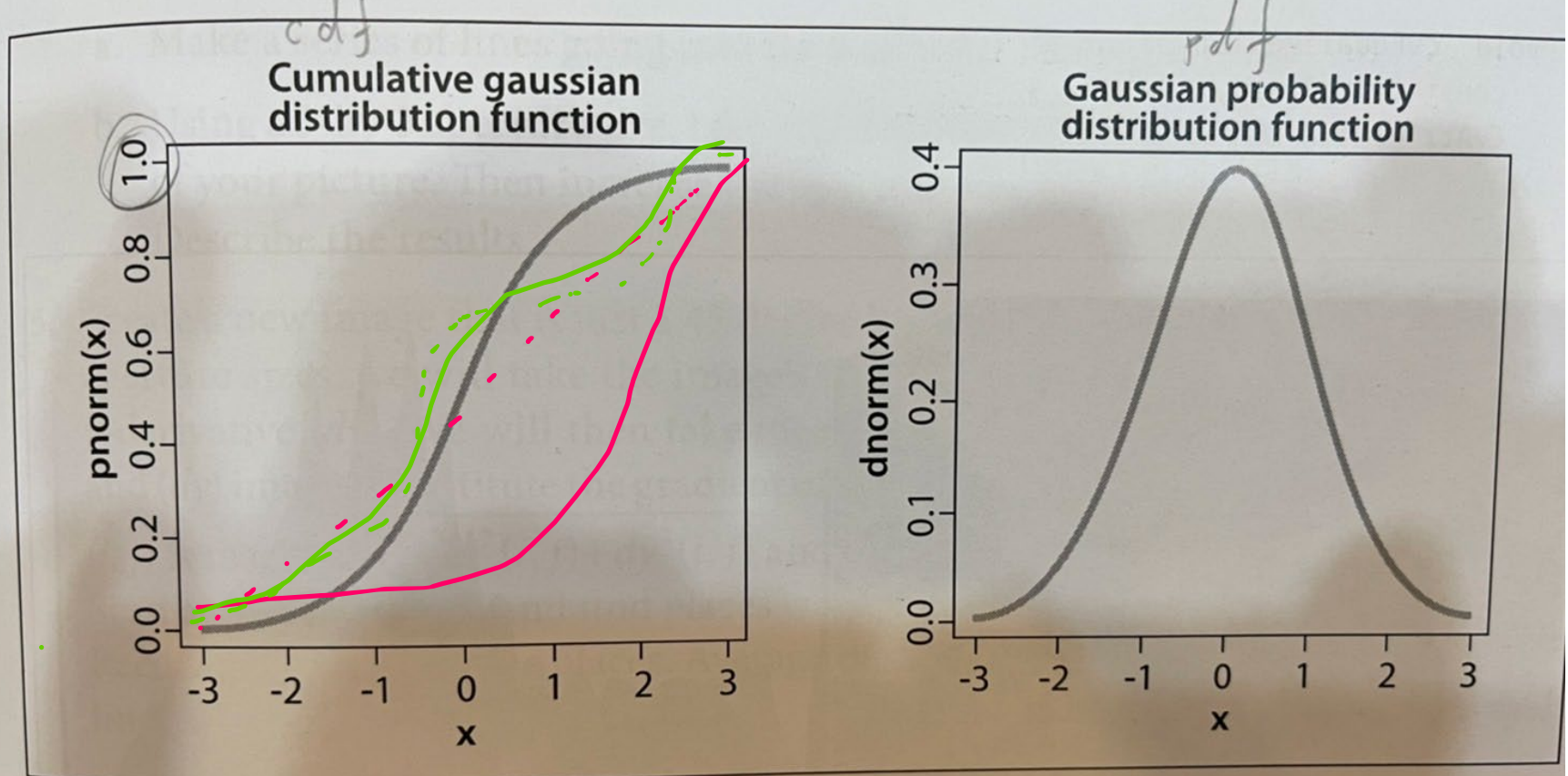


Figure 6-23. Result of cumulative distribution function (left) on a Gaussian distribution (right)

3.1 Histogram Equalization:

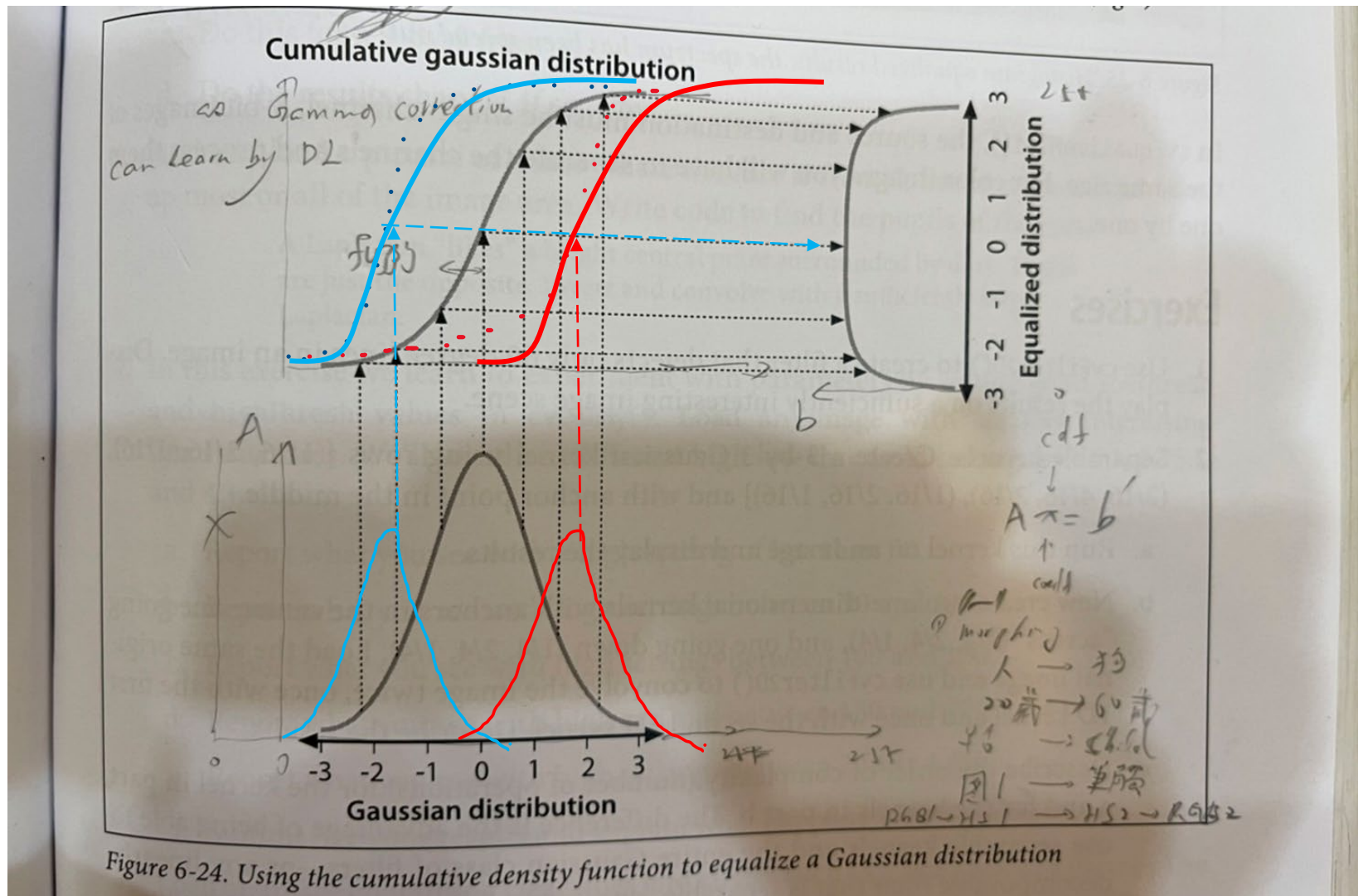


Figure 6-24. Using the cumulative density function to equalize a Gaussian distribution

3.1 Histogram Equalization:

```
void cvEqualizeHist(  
    const CvArr* src,  
    CvArr*      dst  
);
```

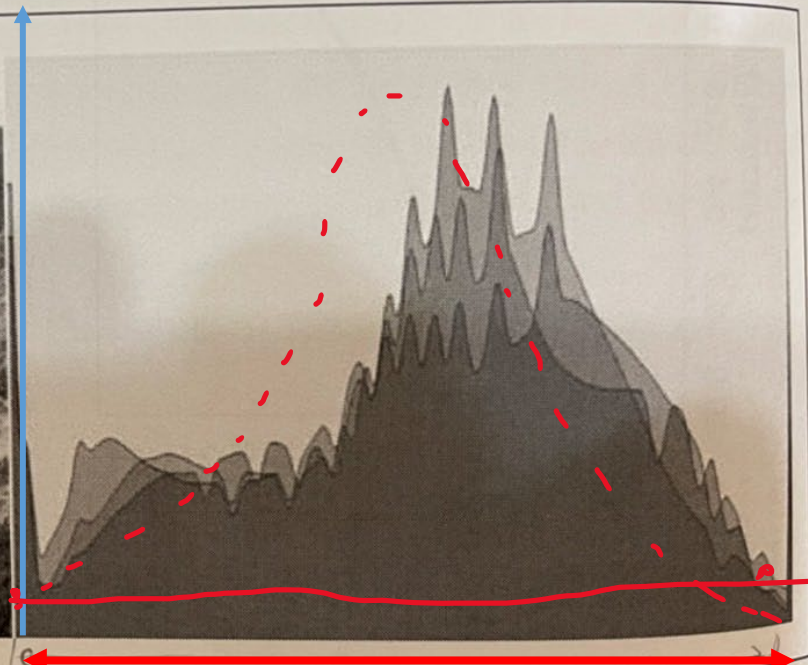
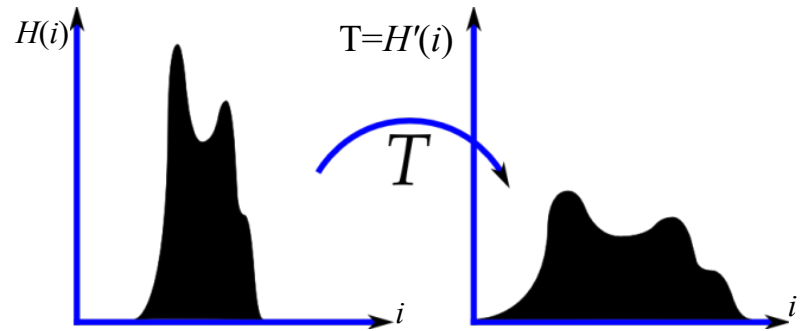


Figure 6-25. Histogram equalized results: the spectrum has been spread out

dynamic range

3.2 Histogram Equalization: Formula (1/2) ???

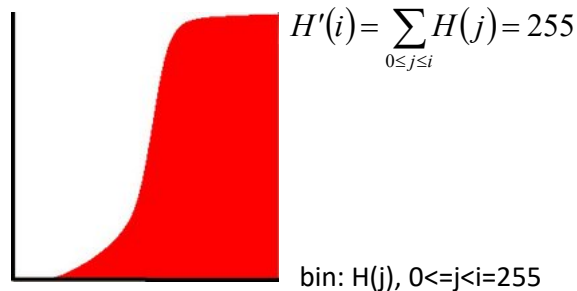


◆ How does it work?

1. To accomplish the equalization effect, the remapping should be the **cumulative distribution function** (cdf). For the histogram $H(i)$, its cumulative distribution $H'(i)$ is:

$$H'(i) = \sum_{0 \leq j \leq i} H(j)$$

2. To use this as a remapping function, we have to **normalize $H'(i)$ such that the maximum value is 255** (or the maximum value for the intensity of the image). From the example above, the cumulative function is:



??

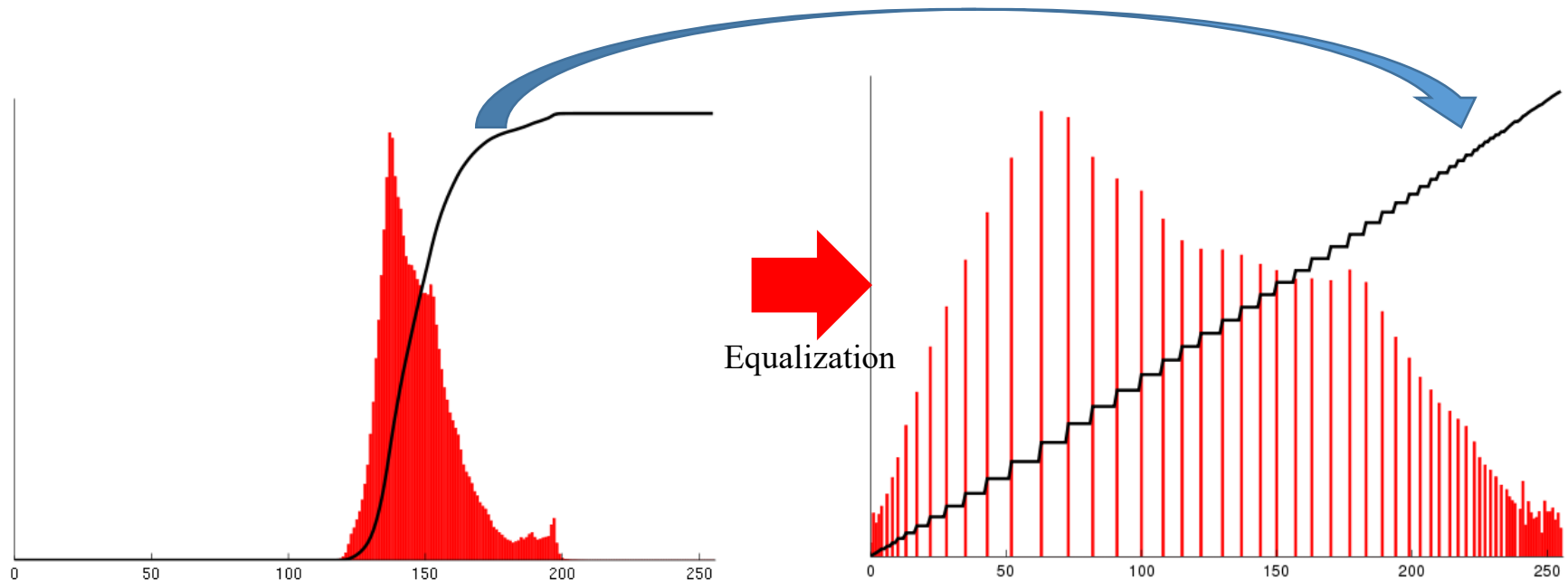
The cumulative function

3.2 Histogram Equalization: Formula (2/2) ??2

3. Finally, we use a simple remapping procedure to obtain the intensity values of the equalized image:

??

$$\text{equalized}(x, y) = H'(\text{src}(x, y))$$



Corresponding histogram (red) and cumulative histogram (black)

3.2 Histogram Equalization: CDF

3.2 Histogram Equalization: Function Call ???

◆ Equalizes the histogram of a grayscale image

1) `equalizeHist`

(1) C++: `void equalizeHist(InputArray src, OutputArray dst)`

2) Parameters

(1) `src` – Source 8-bit single channel image.

(2) `dst` – Destination image of the same size and type as `src`.

3) The function equalizes the histogram of the input image using the following algorithm:

(1) Calculate the histogram H for `src`.

(2) ?? Normalize the histogram so that the sum of histogram bins is 255.

(3) Compute the integral of the histogram: $H'(i) = \sum_{0 \leq j \leq i} H(j)$

(4) Transform the image using $H'(\text{cdf})$ as a look-up table: $dst(x, y) = H'(src(x, y))$

4) The algorithm normalizes the brightness and increases the contrast of the image.

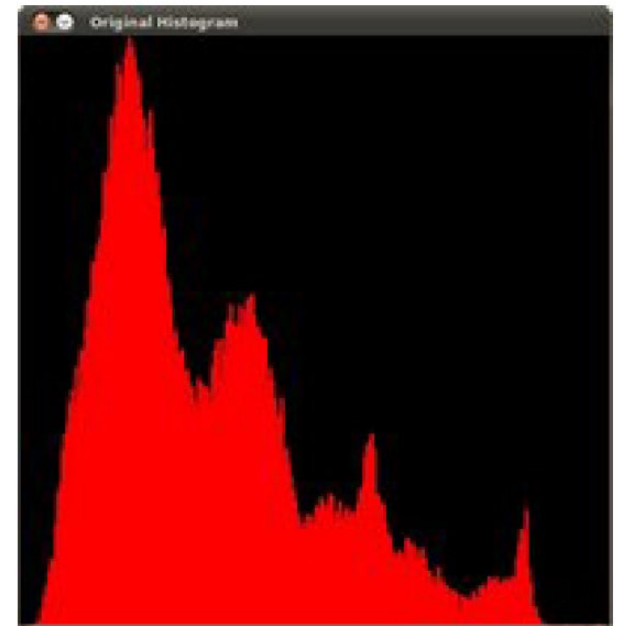
3.2 Histogram Equalization: Result 1 (1/2)

◆ Result 1

1. To appreciate better the results of equalization, let's introduce an image with not much contrast, such as:



which, by the way, has this histogram:



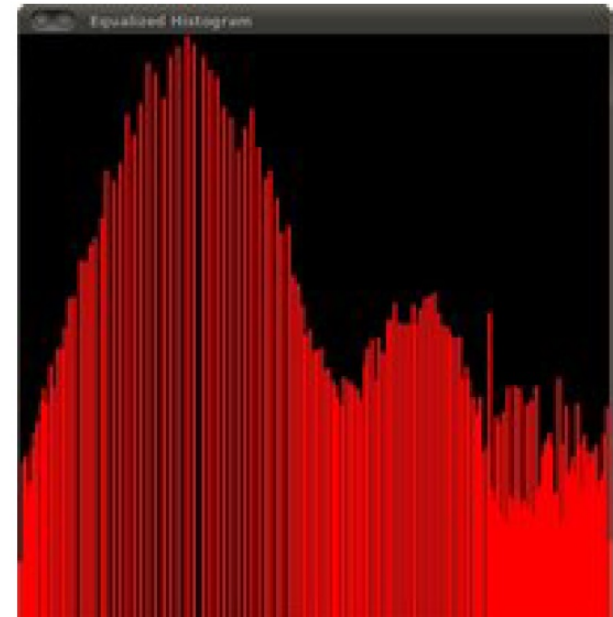
notice that the pixels are clustered around the center of the histogram.

3.2 Histogram Equalization: Result 1 (2/2)

2. After applying the equalization with our program, we get this result:



this image has certainly more contrast. Check out its new histogram like this:



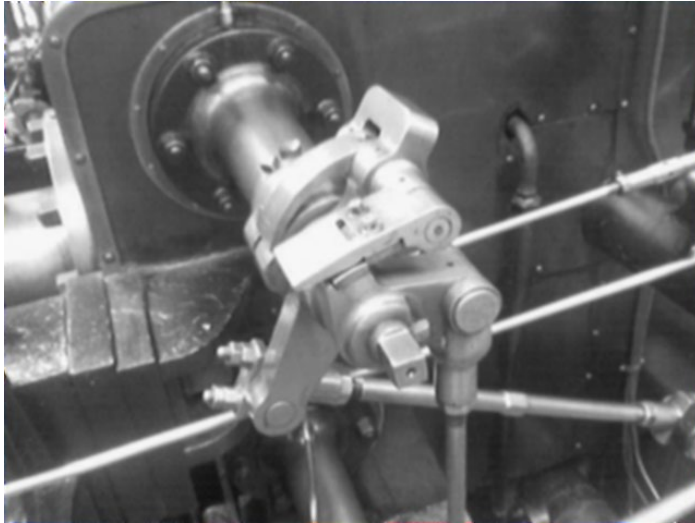
Notice how the number of pixels is more distributed through the intensity range.

Note: Are you wondering how did we draw the Histogram figures shown above? Check out the following tutorial!

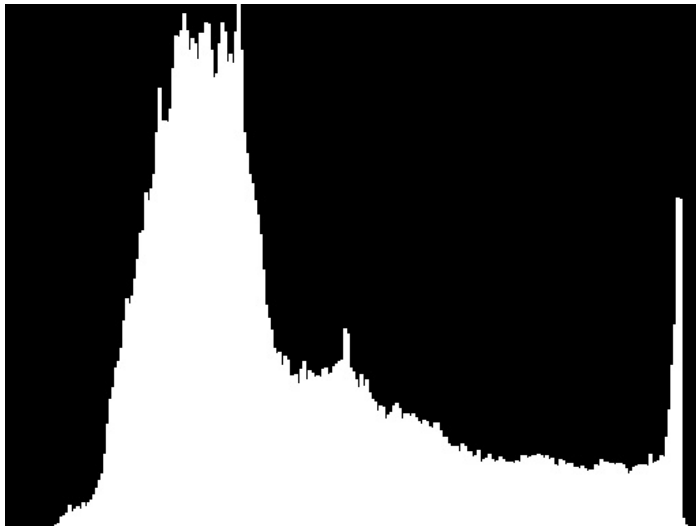
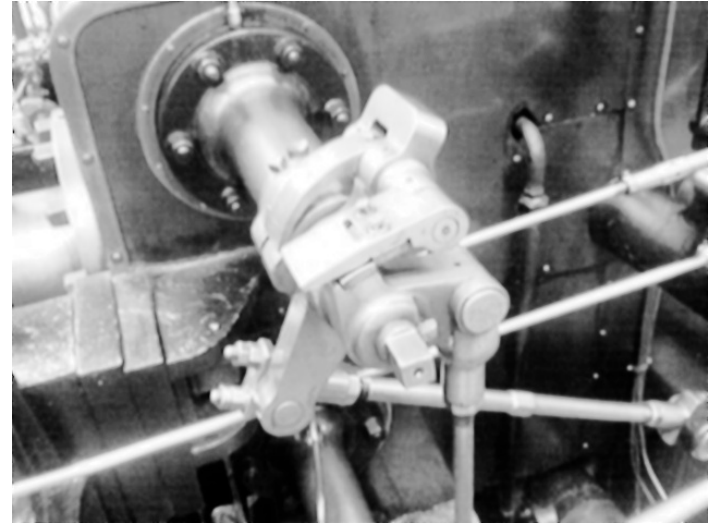
3.2 Histogram Equalization: Result 2

◆ Result 2

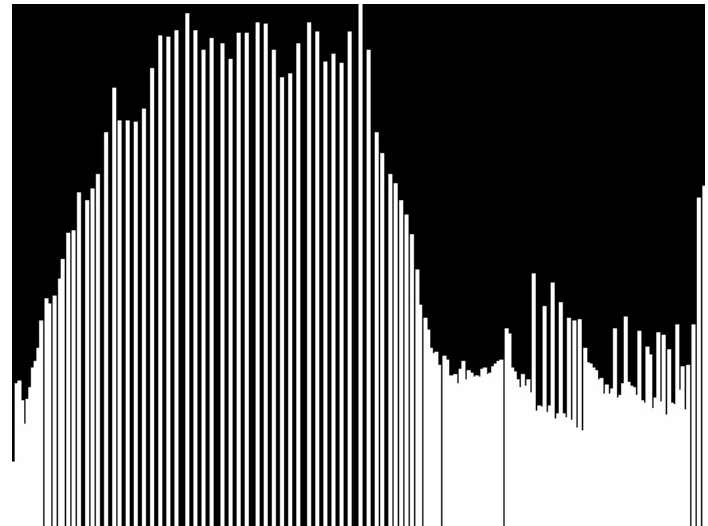
Input image



Output image



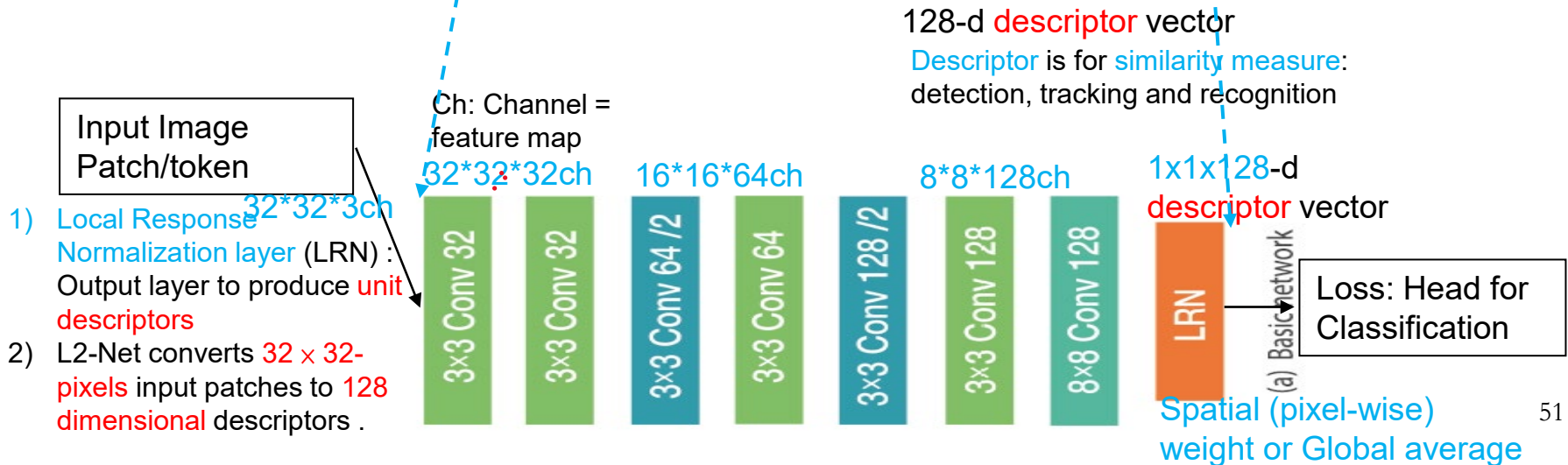
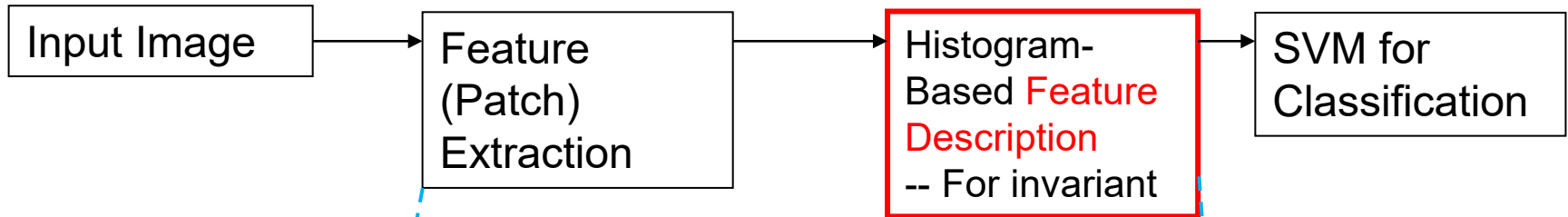
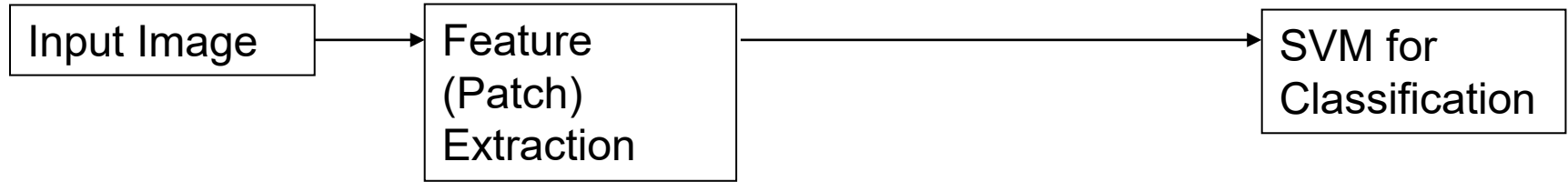
Histogram of input image



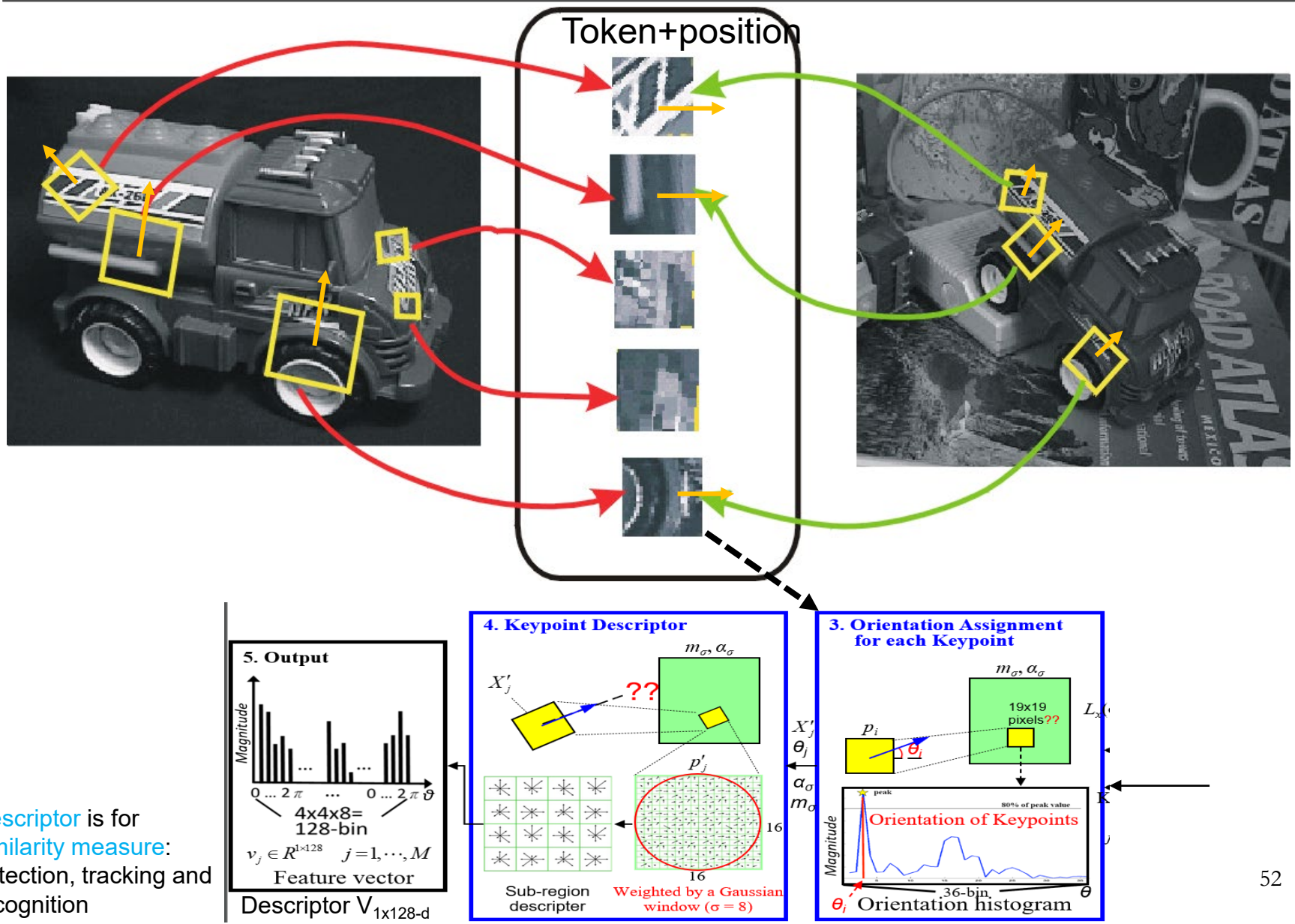
Histogram of output image

3.3 Histogram Application: SIFT Descriptor

CV-SIFT Vs. Deep Learning



3.3 Histogram Application: SIFT Descriptor

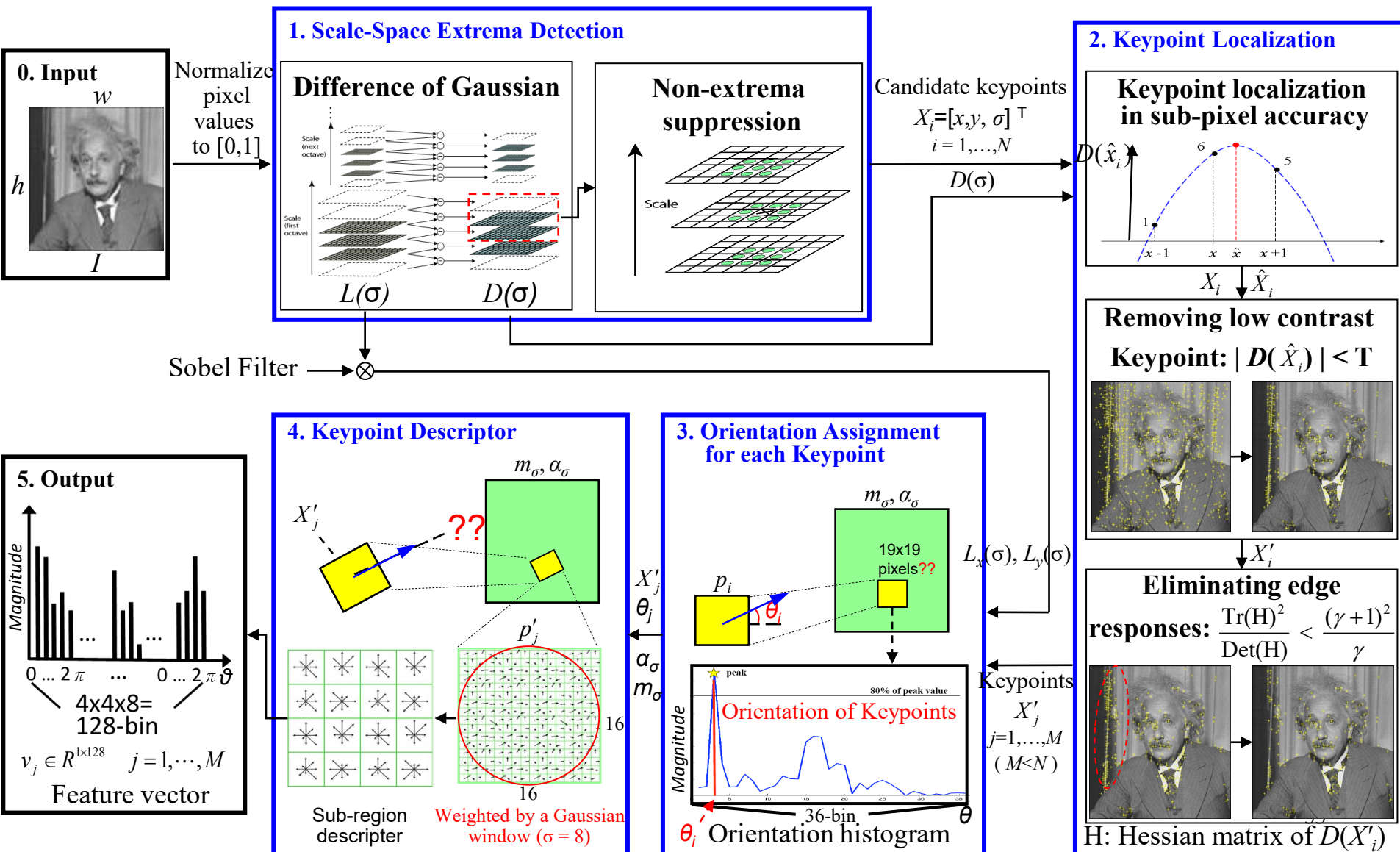


System Flowchart

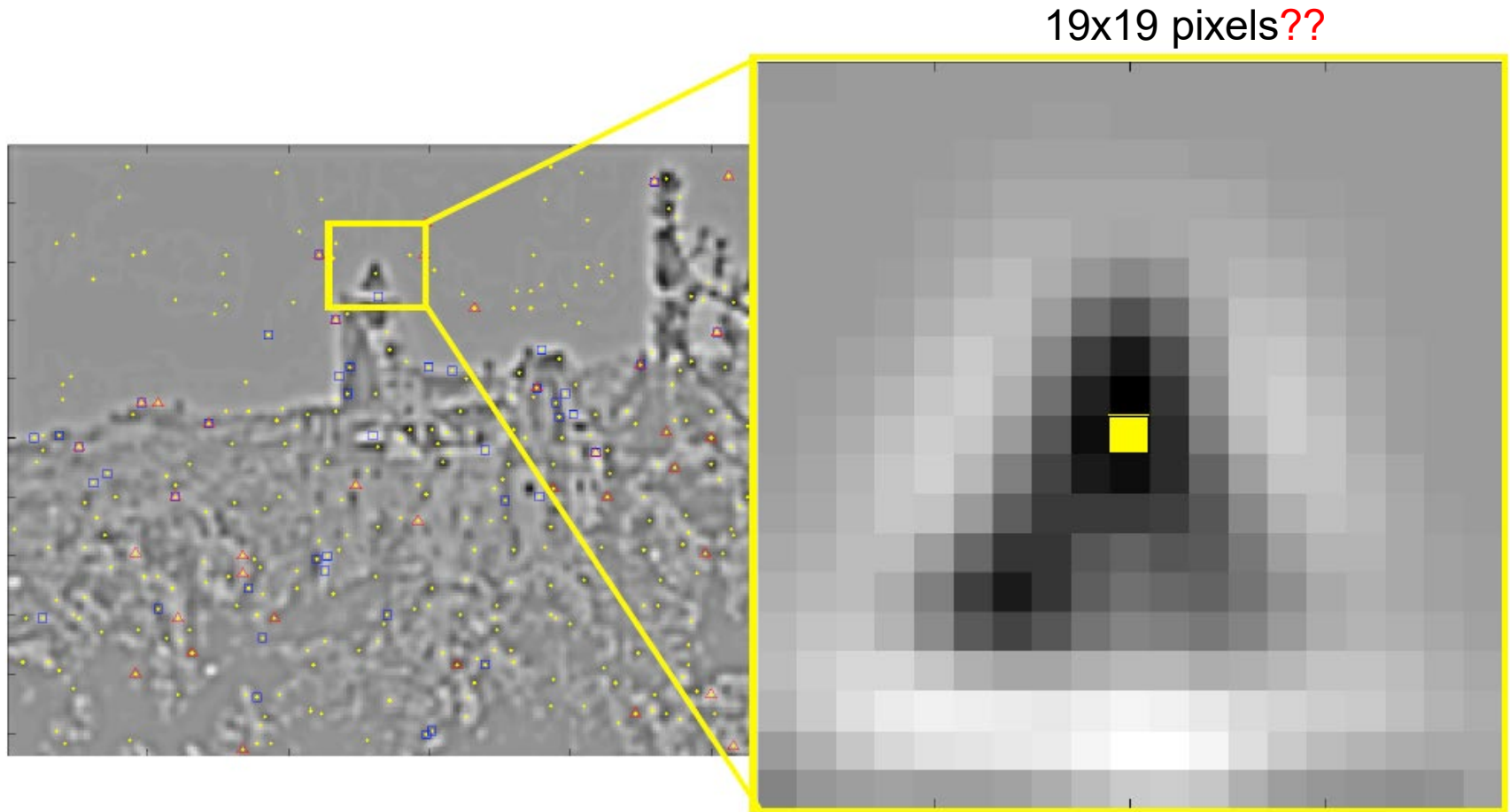
$$L(\sigma) = G(\sigma) * I$$

$$D(\sigma) = (G(k^{(n-1)}\sigma) - G(k^n\sigma)) * I$$

(Scale space) α_σ (Gradient orientation of $L(\sigma)$)
 (DoG images) m_σ (Gradient magnitude of $L(\sigma)$)
 X'_i (Accurate keypoint localization)
 θ_j (Orientation of i -th keypoint)



3.3.1 Orientation Assignment



- Keypoint location = extrema location
- Keypoint scale is scale of the DOG image

Previously, L is I after Gaussian smooth:

Window size??, $\tan_Theta = 180$

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

3.3.1 Orientation Assignment ??

P_j : j -th patch
 $m(x, y)$: gradient magnitude
 $a(x, y)$: orientation
 P_j^m : Gradient magnitude of P_j
 P_j^a : Gradient orientation of P_j
 $L(\sigma)$: Scale space of image.

- Each sample added to the histogram is weighted by its **gradient magnitude** and by a **Gaussian-weighted circular window**.

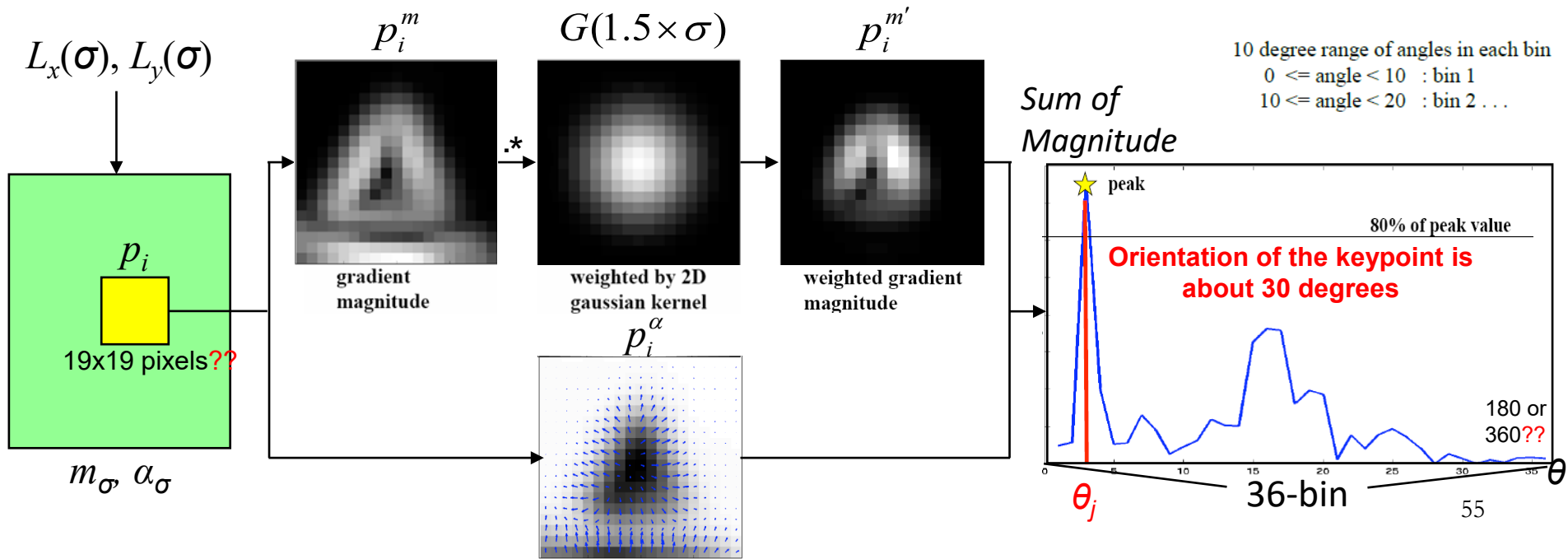
$$L_x(x, y, \sigma) = L(x+1, y, \sigma) - L(x-1, y, \sigma) \Rightarrow m_\sigma(x, y) = \sqrt{L_x(x, y, \sigma)^2 + L_y(x, y, \sigma)^2}$$

$$L_y(x, y, \sigma) = L(x, y+1, \sigma) - L(x, y-1, \sigma) \quad \downarrow \quad \alpha_\sigma(x, y) = \tan^{-1} \left(\frac{L_y(x, y, \sigma)}{L_x(x, y, \sigma)} \right)$$

$$L_x(\sigma) = \begin{bmatrix} -1 & 0 & +1 \\ x-1 & x & x+1 \end{bmatrix} * L \quad L_y(\sigma) = \begin{bmatrix} -1 & 0 & +1 \\ y-1 & y & y+1 \end{bmatrix} * L$$

➤ Consistent orientation for each feature is θ_j .

$$p_j^{m'}(x, y) = p_j^m(x, y) \times G(x, y, 1.5 \times \sigma)$$

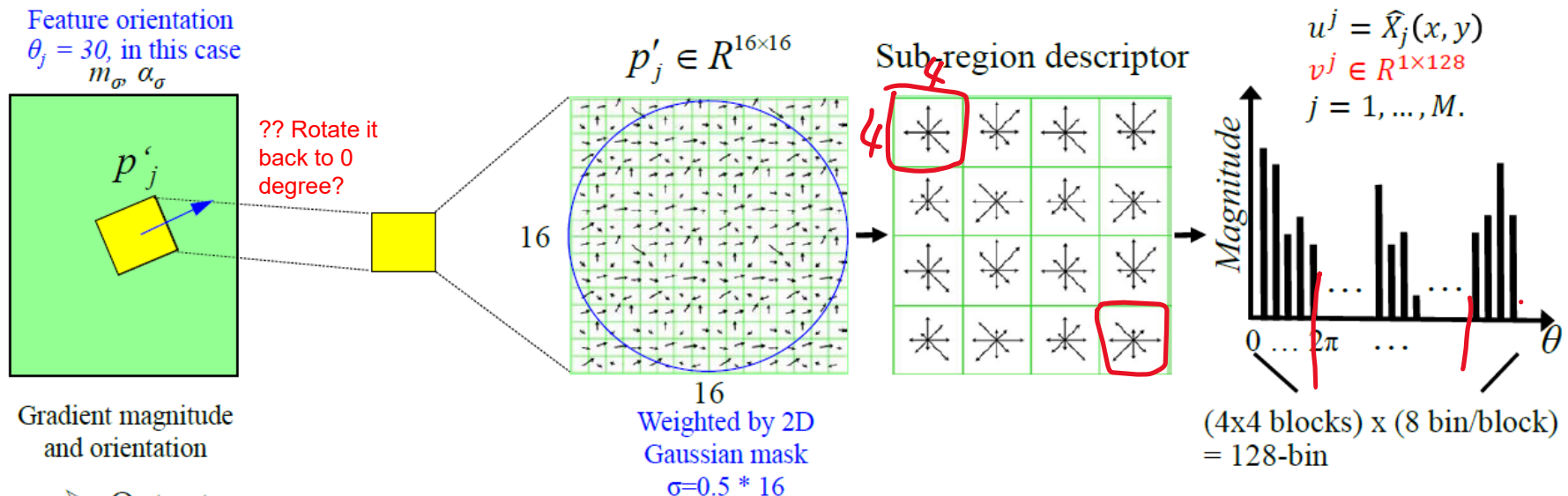


3.3.2 Keypoint Descriptor ??

4) Compute feature descriptor.

- Image gradients are sampled over 16x16 array of locations in scale space relative to the keypoint orientation.

- The feature vector, v_j (128 dimensions), is normalized to unit length. (Reduce change contrast) $v_j \rightarrow \frac{v_j}{\|v_j\|}$



- Output:

- Each image has M features.
 - ✓ Each feature has 128 dimension feature vector.
 - ✓ The center position of each feature.

$u_n^j = [x, y]$: The center position of feature j in image n.

$v_n^j \in R^{1 \times 128}$: Feature vector j in image n (Feature Descriptor).

$k = 1, \dots, M.$

3.3.3 Application: Image Retrieval



• • •
> 5000
images

change in viewing angle
+ scale change



Content: Topic1. Similarity Measure (相似度量測)

1.0 Computer Vision - Camera Model: Between 3D and 2D

1.1 2D Image

1.2 Camera Model Btw 3D Object and 2D Image

1.3 From Linear Combination to Homography $Ax=b$ (Vector, Matrix)

2.0 Computer Vision (CV) and AI: Similarity Measure = Loss Function

2.1 Sum-of-Squared Differences (SSD) and Mean Squared Error (MSE)

2.2 Normalized Correlation Coefficient (NCC) and Cross Entropy (CE)

2.3 AI Bayesian Decision Rule - Maximum a Posteriori (MAP) Probability with Gaussian Model

3.0 Image Processing for Deep Learning: Filter, Correlation and Convolution

3.1 Low-Pass Filter - Denoise and High-Pass Filter - Edge Detection

3.2 Correlation Vs. Convolution

3.3 Multi-Resolution - Downsampling (Encoder) and Upsampling (Decoder)

4.1 Template Matching: Formula

$$\min SSD = \min(T - I)^2 = \min(T^2 - 2TI + I^2)$$

$$\min SSD \rightarrow (T^2 + I^2) - 2(TI)$$

$$\therefore \max CC \rightarrow TI$$

Cross Correlation & Correlation Coefficient

4.1 Template Matching: Goal and Theory **jj2**

Goal

In this tutorial you will learn how to:

- Use the OpenCV function `matchTemplate` to search for matches between an image patch and an input image
- Use the OpenCV function `minMaxLoc` to find the maximum and minimum values (as well as their positions) in a given array.

Location/position

Theory

What is template matching?

Template matching is a technique for finding areas of an image that match (are similar) to a template image (patch).

➤ **Template Matching = Similarity Measure**

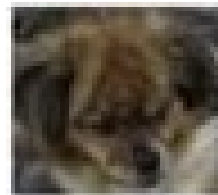
4.1 Template Matching: Definition (1/3) **jj2**

◆ How does it work? Match procedure:

1. Template Image and Source Image Collection: We need two primary components-
 - 1) **Source image (I)**: The image in which we expect to find a match to the template image
 - 2) **Template image (T)**: The **patch** image which will be compared to the template image



source image



template image



the highest matching region

4.1 Template Matching: Definition (2/3) jj2

Template matching as convolution

◆ How does it work?

2. Template Search Sliding along Scanline:

- To identify the matching area, we have to compare the template image against the source image by **sliding** it:



Source Image



Template (patch)



Compare the template image against the source image by sliding it.

Compare the template image against the source image by sliding it.

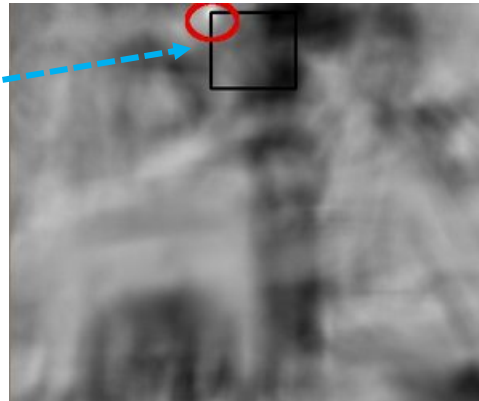
- By **sliding (scanline)**: we mean moving the patch one pixel (**called stride 1 pixel**) at a time (**left to right, up to down**). At each location, a **metrics** is calculated so it represents how “**good**” or “**bad**” the match at that location is (or **how similar** the patch is to that particular area of the source image).

4.1 Template Matching: Definition (3/3) **jj2**

3. **Matching Metrics** Matrix Image R Creation:

- For each location of T over I , you store the **metrics** in the result matrix (R). Each location (x,y) in R contains the match metric:

Sliding window w
 $w(w',h')$



$R(x,y)$: The brightest location =
the best matching location:

- Top-left corner of T , or
- Center of T

Usually it will locate at
patch center

- The image is the result R of sliding the patch with a metric.
- The brightest locations indicate the best matching location.

4. Matching Location Decision:

- As you can see, the location marked by the red circle is **probably** the one with the **best matching value**, so that location (the rectangle formed by that point as a corner and width and height equal to the patch image) is considered the match.

4.2 Match Metric: SSD and MSE – SSD

1. SSD: Sum-of-Squared Differences

- OpenCV implements template matching in the function matchTemplate:

1.1 CV_TM_SQDIFF: Sum-of-Squared-Differences (SSD)

$$\min R(x, y) = \sum_{\substack{x', y' \in w \\ x', y' \text{ belongs to} \\ \text{window } w}} (T(x', y') - I(x + x', y + y'))^2$$

Sliding window w
 $w(x', y')$: $x' \in 0 \sim (w'-1)$,
 $y' \in 0 \sim (h'-1)$

Window $w(x', y')$
 Slide/shift/stide (x, y) over
 entire source image

$$\min R(x, y) = \sum_{x'=0}^{h'-1} \sum_{y'=0}^{w'-1} [(T(x', y')) - I(x + x', y + y')]^2$$

1.2 CV_TM_SQDIFF_NORMED: Normalized Sum-of-Squared Differences

$$\min R(x, y) = \frac{\sum_{x', y'} (T(x', y') - I(x + x', y + y'))^2}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$$

1.3 SQDIFF_NORMED – Normalized by Gaussian Model (Variance)

m : Mean or
average value

$$\min R(x, y) = \frac{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(T(x', y') - m^T) - I(x + x', y + y') - m^I]^2}{\sqrt{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(T(x', y') - m^T)]^2} \sqrt{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(I(x + x', y + y') - m^I)]^2}}$$

σ^2 : Standard deviation

4.2 Match Metric: SSD and MSE – MSE

2. MSE: Mean Squared Error

Loss Function:

$$L_b = 1/N * \sum_{n=1}^N \{ \sum_{x'=0}^{h'-1} \sum_{y'=0}^{w'-1} [T(x', y') - I(x', y')]^2 \}$$
$$= 1/N * \sum_{n=1}^N \{ \sum_{u=0}^h \sum_{v=0}^w [y(u, v) - y'(u, v)]^2 \}$$

b : b -th batch

N : Total number of
images or Batch size

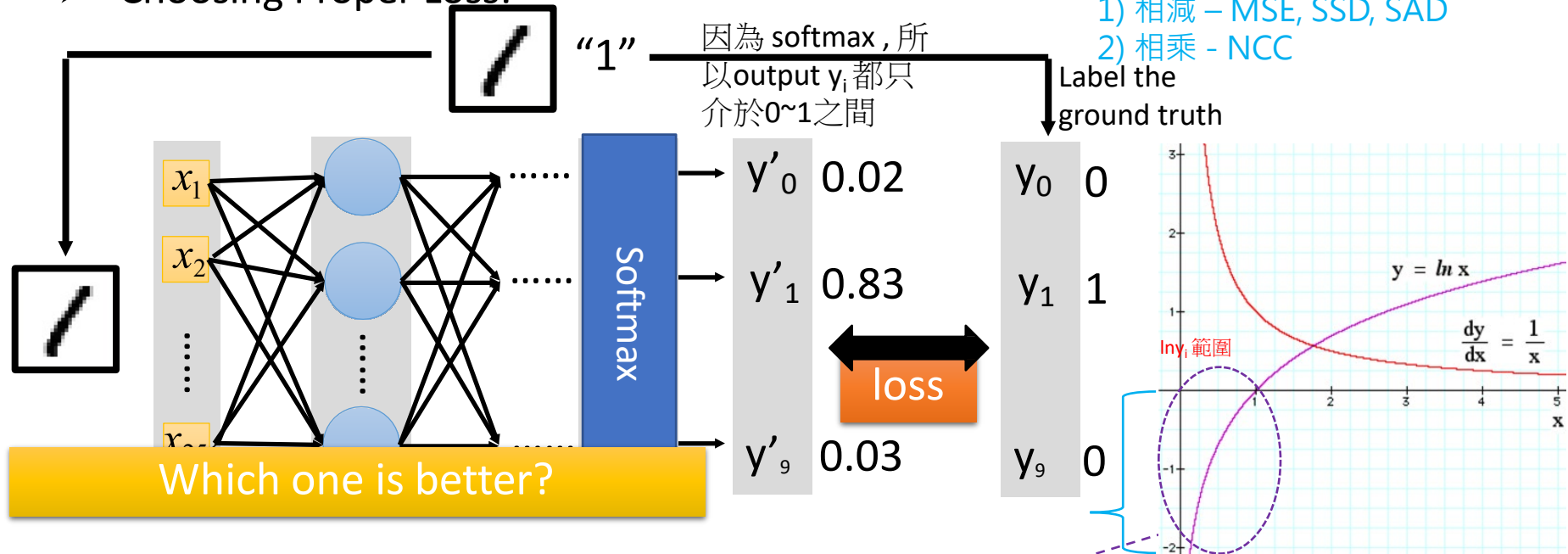
y : Ground
truth values

y' : Prediction or
estimation values

4.2 Match Metric: MSE Vs. CE – Loss Function

3. Loss Function at Deep Learning – 1) MSE and 2) Cross Entropy

➤ Choosing Proper Loss:



1) Mean Square Error: for continuous data, like detection Bounding Box (Bbox)

$$L_b = 1/N * \sum_{n=1}^N \{ \sum_{i=0}^9 (y_i - y'_i)^2 \}$$

Why not L1 | | ➔ why L2 Or Smooth L1

Not continuous, cannot derivative Continuous, can optimize by derivative

2) Cross Entropy: for discrete data, like classification

$$L_b = - \sum_{i=0}^9 \{ y_i \ln y'_i \}, \text{ or } L_b = L_{b1} + L_{b2} \begin{cases} L_{b1} = - \sum_{i=0}^9 \{ y_i \ln y'_i \}, & \text{if } y_i = 1 \\ L_{b2} = - \sum_{i=0}^9 \{ (1 - y_i) \ln y'_i \}, & \text{Otherwise } (y_i = 0) \end{cases}$$

- Before, have Softmax for normalizing all output values, so don't need 1/N

- Because y is prob. 0.0~1.0, so log y will be -, therefore - - = +

Content: Topic1. Similarity Measure (相似度量測)

1.0 Computer Vision - Camera Model: Between 3D and 2D

1.1 2D Image

1.2 Camera Model Btw 3D Object and 2D Image

1.3 From Linear Combination to Homography $Ax=b$ (Vector, Matrix)

2.0 Computer Vision (CV) and AI: Similarity Measure = Loss Function

2.1 Sum-of-Squared Differences (SSD) and Mean Squared Error (MSE)

2.2 Normalized Correlation Coefficient (NCC) and Cross Entropy (CE)

2.3 AI Bayesian Decision Rule - Maximum a Posteriori (MAP) Probability with Gaussian Model

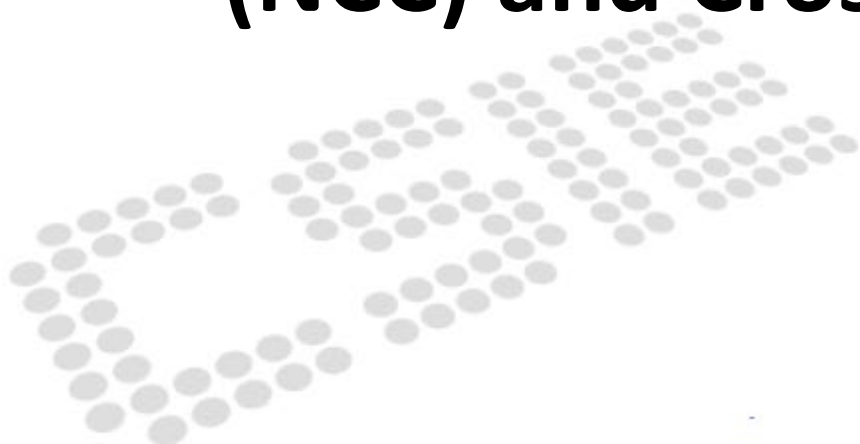
3.0 Image Processing for Deep Learning: Filter, Correlation and Convolution

3.1 Low-Pass Filter - Denoise and High-Pass Filter - Edge Detection

3.2 Correlation Vs. Convolution

3.3 Multi-Resolution - Downsampling (Encoder) and Upsampling (Decoder)

2.2 Normalized Correlation Coefficient (NCC) and Cross Entropy (CE)



4.3.1 From SSD to CC

◆ From sum-of-squared difference (SSD) to cross correlation (CC)

$$\min SSD = \min(T - I)^2 = \min(T^2 - 2TI + I^2)$$

$$\min \cancel{SSD} \rightarrow (T^2 + I^2) - 2(\cancel{TI})$$

$$\therefore \max CC \rightarrow \frac{TI}{T * I}$$

→ Cross Correlation

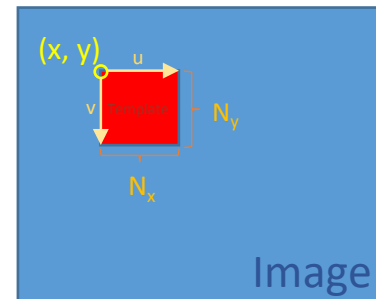
4.3.1 Template Matching + Scanline: NCC (1/2)

1. **Cross Correlation, $R(x, y)$** : The correlation (Linear combination) between two signals (**cross correlation**) is standard approach to **feature detection** as well as a component of more sophisticated techniques. Similarity measure: MSD (minus: -), Cross Entropy (product: x)

$$R(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} (T(u, v) \cdot I(x + u, y + v))$$

← 其中內積是取其投影量來求相似度
inside the product is to take its projection amount to find similarity
← 計算的起始位置在左上角
The starting position of the calculation is at the top left corner

OpenCV



-Source image (I, MxM): The image in which we expect to find a match to the template image

Template image (T, NxN): The patch image which will be compared to the template image

Slide/shift (x, y); Window posited (u, v)

EX: 1x9, 2x8, ... 5x5(Max)

- Problem: $R(x, y)$ range is not btw 0.0~1.0

But where is the threshold? So we got the

Normalization of cross correlation

2. **Normalized Cross Correlation**: But the normalized form of correlation (**normalized correlation coefficient** or **unit vector**) **preferred in template matching does not have a correspondingly simple and efficient frequency domain expression**. For this reason normalized cross-correlation has been computed in the **spatial domain**.

$$R(x, y) = \frac{\sum_{u=0}^{N-1} \sum_{v=0}^{N-1} (T(u, v) \cdot I(x + u, y + v))}{\sqrt{[\sum_{u=0}^{N-1} \sum_{v=0}^{N-1} (T(x, y))^2][\sum_{u=0}^{N-1} \sum_{v=0}^{N-1} (I(x + u, y + v))^2]}}$$

← 沒有減掉mean不是NCC, 只是整理成同樣的scale

Not the NCC, but it's just the same scale.

← 結果也是會被環境光影響

Sam

← Normalized Unity

-Source image (I, MxM): The image in which we expect to find a match to the template image

Template image (T, NxN): The patch image which will be compared to the template image

Slide/shift (x, y); Window posited (u, v)

- $R(x, y)$ range is normalized btw -1.0~1.0

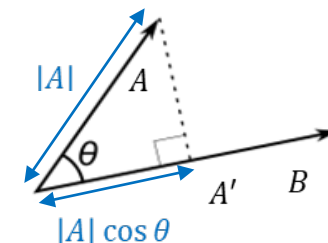
$$\cos \theta = \frac{A^T B}{\|A\| \|B\|}, 0.0 \leq \cos \theta \leq 1.0$$

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}}$$

產生類似餘弦的相關係數

$$\vec{A} \cdot \vec{B} = |A| \cdot |B| \cdot \cos \theta$$

$$\cos \theta = \frac{\vec{A} \cdot \vec{B}}{|A| |B|}$$



4.3.1 Template Matching + Scanline: NCC (2/2)

3. Matric: similarity measure - NCC (Normalized Correlation Coefficient)

- At each location, a **metric** is calculated so it represents how “good” or “bad” the match at that location is (or how similar)
- For each location of t over f , we store the metric in the result matrix R . Each location (x, y) in R contains the match metric:
 - $R(x, y)$ range is normalized btw $-1.0 \sim 1.0$
 - NCC is a Gaussian Model $T(,)$ and $I(,)$

$$\max R(x, y) = \frac{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(T(x', y') - m^T) * I(x + x', y + y') - m^I]^2}{\sqrt{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(T(x', y') - m^T)]^2} \sqrt{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(I(x + x', y + y') - m^I)]^2}}$$



← 起始點在左上角，由左至右，由上至下

The starting point is in the upper left corner, from left to right, from top to bottom

The image is the result γ of sliding the patch with a metric `TM_CCORR_NORMED`.

The brightest locations indicate the highest matches

4. Matching Result:

- One target per source image**: At the image above, the location marked by the red circle is **probably** the one with the highest value, so that the rectangle formed by that point as a corner is considered the match.

- N targets per source image**: Need to consider the **mergence** problem, such as **subpixel** problem can using weighted mergence

→ Non-Maximum Suppression for mergence

← 若出現許多matching score超過threshold的目標，必須選擇一個最大的目標，並整合周圍的目標 by **probability weighting = similarity estimation**。

If there are many goals with matching scores that exceed threshold, you must choose the largest goal and integrate the surrounding goals

4.3.1 Match Metric: NCC and CE - NCC

1,2,...,4,5,6,...8,9
9,8,...,6,5,4,...2,1

1. NCC - Normalized Correlation Coefficient:

1.1 CV_TM_CCORR: Cross Correlation

$$\max R(x, y) = \sum_{x', y' \text{ belong to window } w} (T(x', y') \cdot I(x + x', y + y'))$$

Deep Learning:	min	max
1) MSE	SSD	correlation
2) Cross Entropy	9-1 = 8	9*1 = 9
	5-5 = 0	5*5 = 25

1.2 CV_TM_CCORR_NORMED: Normalized Cross Correlation

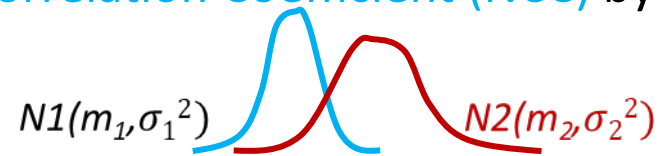
OpenCV

$$\max R(x, y) = \frac{\sum_{x', y'} (T(x', y') \cdot I(x + x', y + y'))}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$$

- Is NCC a Gaussian Model? Yes,
- Is Gaussian model a good model?
- Slow
- Modify to SSD or MSD
- Faster, without normalization
- Batch Normalization

1.3 CV_TM_CCOEFF_NORMED: Normalized Correlation Coefficient (NCC) by Gaussian Model

- NCC Range [-1.0, 1.0]



$$\max R(x, y) = \frac{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(T(x', y') - m^T) * I(x + x', y + y') - m^I]^2}{\sqrt{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(T(x', y') - m^T)]^2} \sqrt{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(I(x + x', y + y') - m^I)]^2}}$$

m : Mean or average value
 σ^2 : Standard deviation

- NCC Range [0.0, 1.0] if $|R(x, y)|$, for similarity Measure
Similar measure btw two Gaussian distributions

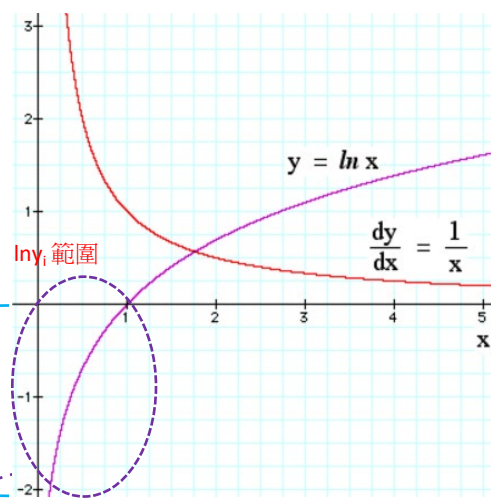
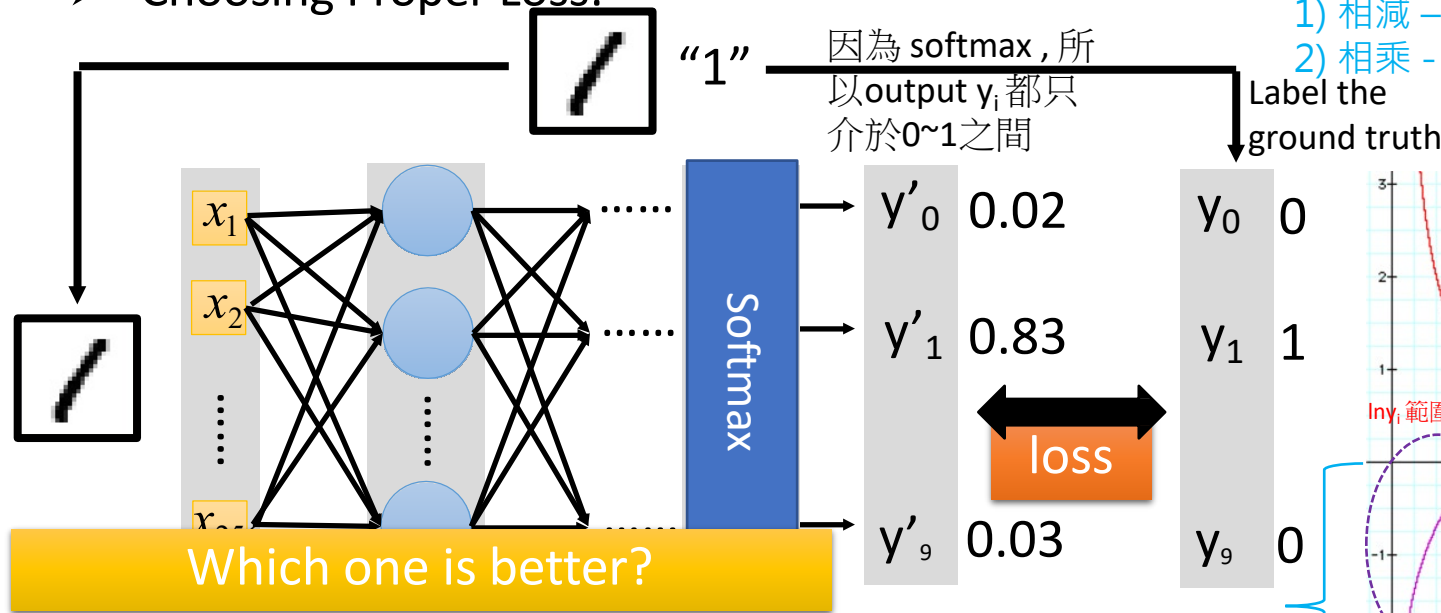
4.3.1 Match Metric: MSE Vs. CE – Loss Function

2. CE – Cross Entropy

3. Loss Function at Deep Learning – 1) MSE and 2) Cross Entropy

相似度判斷(similarity measure):
1) 相減 – MSE, SSD, SAD
2) 相乘 – NCC

➤ Choosing Proper Loss:



1) Mean Square Error: for continuous data, like detection Bounding Box (Bbox)

$$L_b = 1/N * \sum_{n=1}^N \{ \sum_{i=0}^9 (y_i - y'_i)^2 \}$$

Why not L1 | | ➔ why L2 Or Smooth L1
Not continuous, cannot derivative Continuous, can optimize by derivative

2) Cross Entropy: for discrete data, like classification

$$L_b = - \sum_{i=0}^9 \{ y_i \ln y'_i \}, \text{ or } L_b = L_{b1} + L_{b2} \begin{cases} L_{b1} = - \sum_{i=0}^9 \{ y_i \ln y'_i \}, & \text{if } y_i = 1 \\ L_{b2} = - \sum_{i=0}^9 \{ (1 - y_i) \ln y'_i \}, & \text{Otherwise } (y_i = 0) \end{cases}$$

- Before, have Softmax for normalizing all output values, so don't need 1/N
- Because y is prob. 0.0~1.0, so log y will be -, therefore - - = +

4.3.2 Template Matching

Deep Learning:

1) MSE

2) Cross Entropy

Why not L1 | → why L2 Or Smooth L1

Formula (1/5)

min SSD

9-1=8

5-5=0

max correlation

9*1=9

5*5=25

1,2,...,4,5,6,...8,9

9,8,...,6,5,4,...2,1

Not continuous,
cannot derivative

Continuous, can
optimize by derivative

◆ Which are the matching methods available in OpenCV?

OpenCV implements Template matching in the function `matchTemplate`. The available methods are 6 ??:

1.1 CV_TM_SQDIFF: Sum of Squared Differences (SSD)

$$\min R(x, y) = \sum_{\substack{x', y' \in w \\ x', y' \text{ belongs to window } w}} (T(x', y') - I(x + x', y + y'))^2$$

Slide/shift (x,y)
Window (x',y')

1.2 CV_TM_SQDIFF_NORMED: Normalized Sum of Squared Differences

$$\min R(x, y) = \frac{\sum_{x', y'} (T(x', y') - I(x + x', y + y'))^2}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$$

4.4.2 Template Matching: Formula (2/5)

2.1 CV_TM_CCORR: Cross Correlation

$$\max R(x, y) = \sum_{\substack{x', y' \\ x', y' \text{ belongs to window } w}} (T(x', y') \cdot I(x + x', y + y'))$$

2.2 CV_TM_CCORR_NORMED: Normalized Cross Correlation:

$$\max R(x, y) = \frac{\sum_{x', y'} (T(x', y') \cdot I(x + x', y + y'))}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$$

x', y' belongs to window w

4.4.2 Template Matching: Formula (3/5)

2.3 CV_TM_CCOEFF: Correlation Coefficient

$$\max R(x, y) = \max \sum_{x', y'} (T'(x', y') \cdot I'(x + x', y + y'))$$

$$= \max \sum_{\substack{x', y' \\ x', y' \text{ belongs to window } w}} ((T(x', y') - \bar{T}(x', y')) \cdot (I(x + x', y + y') - \bar{I}(x + x', y + y')))$$

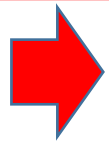
$$\text{where } T'(x', y') = T(x', y') - \frac{1}{(w \cdot h)} \sum_{x'', y''} T(x'', y'')$$

$$I'(x + x', y + y') = I(x + x', y + y') - \frac{1}{(w \cdot h)} \sum_{x'', y''} I(x + x'', y + y'')$$

4.4.2 Template Matching:

Formula (4/5)

Similar measure btw two Gaussian distributions



2.4 CV_TM_CCOEFF_NORMED: Normalized Correlation Coefficient (NCC): $0.0 \leq R(x,y) \leq 1.0$

$$\begin{aligned} \max R(x, y) &= \frac{\sum_{x', y'} (T'(x', y') \cdot I'(x + x', y + y'))}{\sqrt{\sum_{x', y'} T'(x', y')^2 \cdot \sum_{x', y'} I'(x + x', y + y')^2}} \\ &= \frac{\sum_{x', y'} ((T(x', y') - \bar{T}(x', y')) \cdot (I(x + x', y + y') - \bar{I}(x + x', y + y'))) }{\sqrt{\sum_{x', y'} (T(x', y') - \bar{T}(x', y'))^2 \cdot \sum_{x', y'} (I(x + x', y + y') - \bar{I}(x + x', y + y'))^2}} \end{aligned}$$

4.4.2 Template Matching: Function Call (1/2)

1. Compares a template against overlapped image regions

1) matchTemplate

(1) C++: `void matchTemplate(InputArray image, InputArray templ, OutputArray result, int method)`

2) Parameters

- (1) image – Image where the search is running. It must be 8-bit or 32-bit floating-point.
- (2) templ – Searched template. It must be not greater than the source image and have the same data type.
- (3) result – Map of comparison results R. It must be single-channel 32-bit floating-point. If image is $W \times H$ and templ is $w \times h$, then result is $(W - w + 1) \times (H - h + 1)$, (total number of results showing on result image R)
- (4) method – Parameter specifying the comparison method.

4.4.2 Template Matching: Function Call (2/2)

- 3) The function slides through image, compares the overlapped patches of size $w \times h$ against templ using the specified method and stores the comparison results in result.
- 4) After the function finishes the comparison, the best matches can be found as global **minimums (when CV_TM_SQDIFF was used)** or **maximums (when CV_TM_CCORR or CV_TM_CCOEFF was used)** using the **minMaxLoc()** function to the min and max values from [range R](#).

4.4.2 Template Matching: Result 1 (1/3)

- Result 1

1. Testing our program with an input image such as:

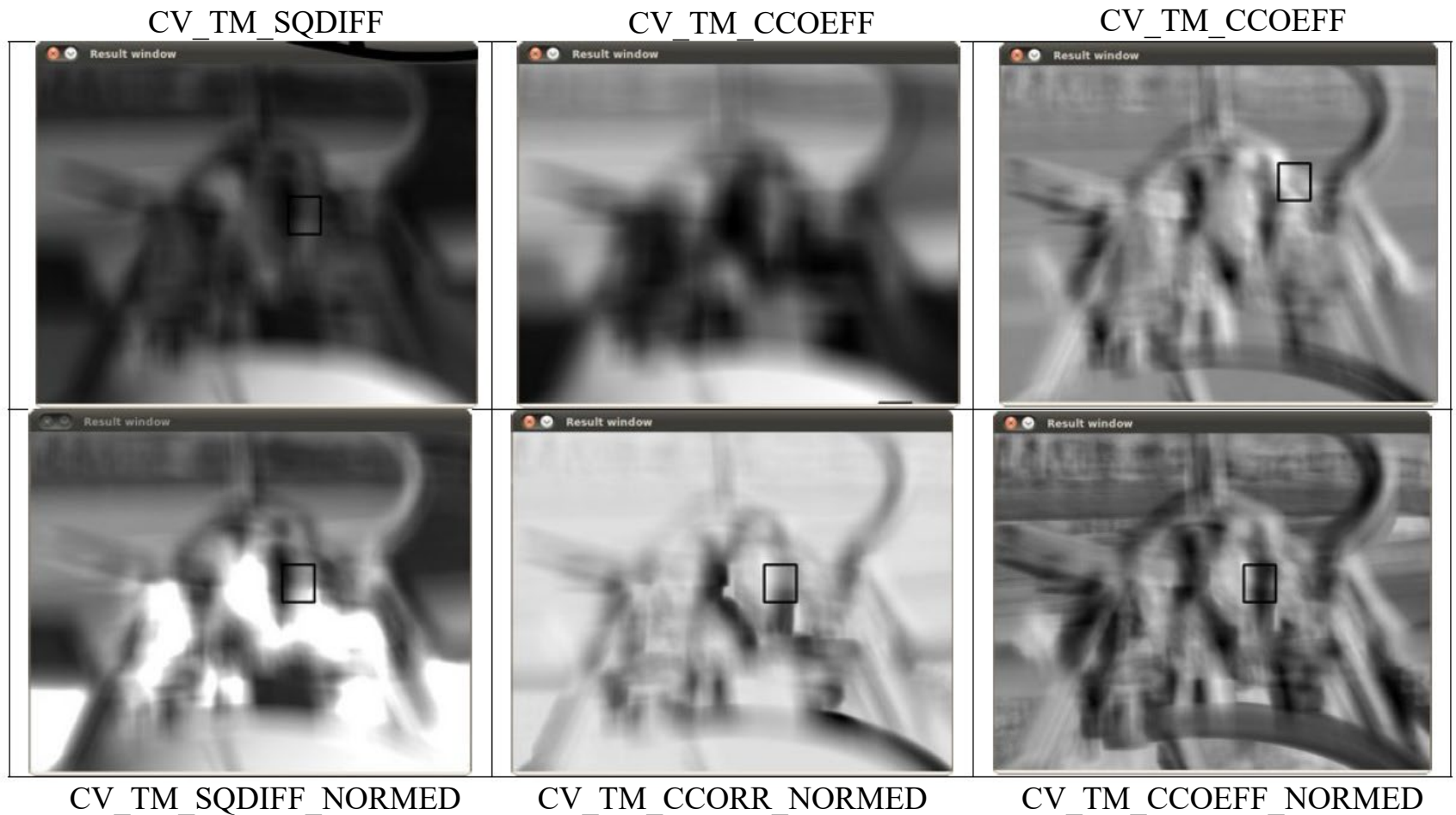


and a template image:



4.4.2 Template Matching: Result 1 (2/3)

2. Generate the following result matrices (first row are the standard methods SQDIFF, CCORR and CCOEFF, second row are the same methods in its normalized version). In the first column, the darkest is the better match, for the other two columns, the brighter a location, the higher the match.



4.4.2 Template Matching: Result 1 (3/3)

3. The right match is shown below (black rectangle around the face of the guy at the right). Notice that CCORR and CCDEFF gave erroneous best matches, however their normalized version did it right, this may be due to the fact that we are only considering the “highest match” and not the other possible high matches.



4.4.2 Template Matching: Result 2 (1/4)

◆ Result 2



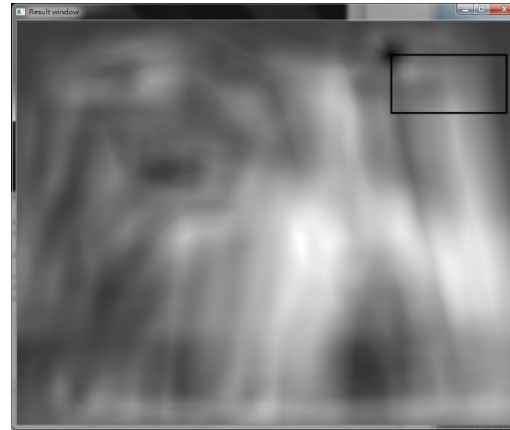
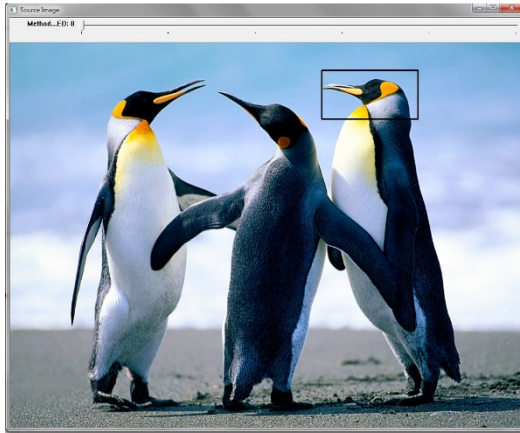
Source image



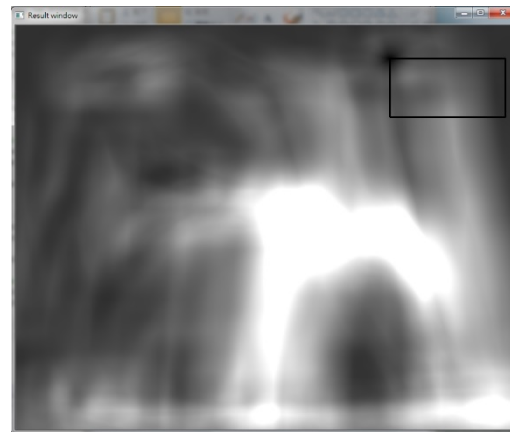
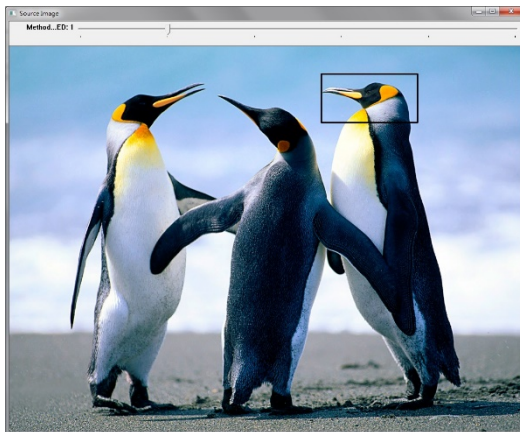
Template image

4.4.2 Template Matching: Result 2 (2/4)

- Method: CV_TM_SQDIFF

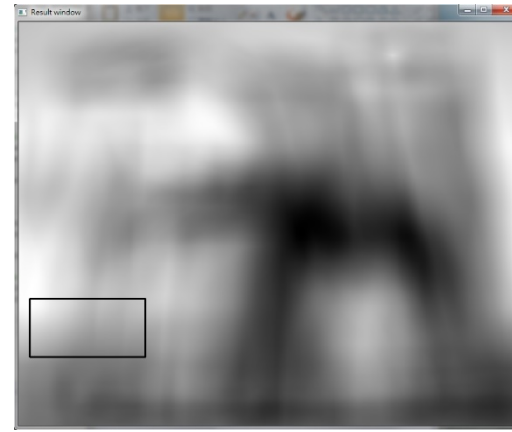


- Method: CV_TM_SQDIFF_NORMED

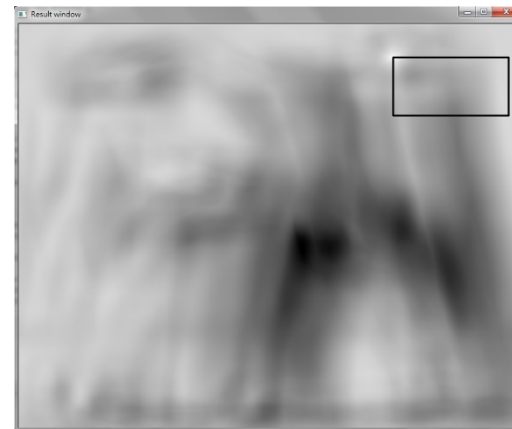
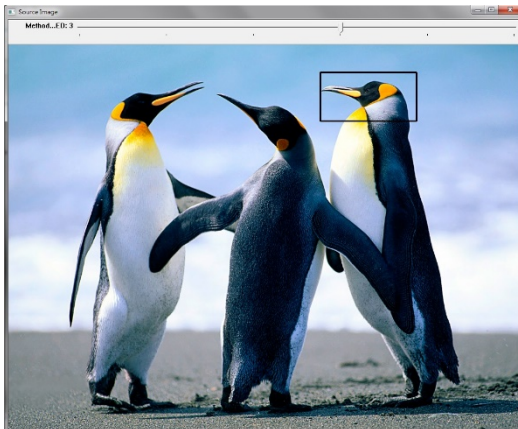


4.2.2 Template Matching: Result 2 (3/4)

- Method: CV_TM_CCORR

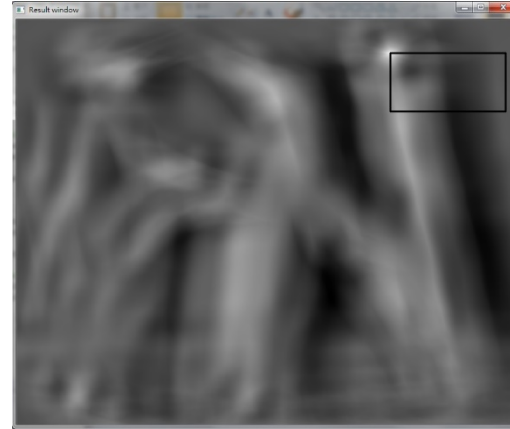
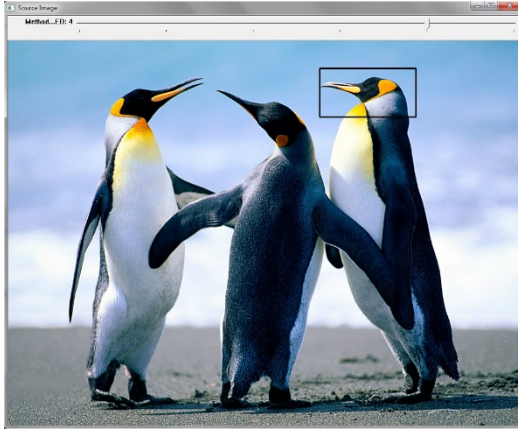


- Method: CV_TM_CCORR_NORMED

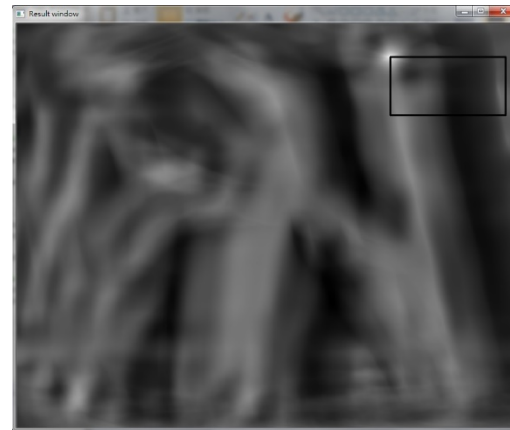
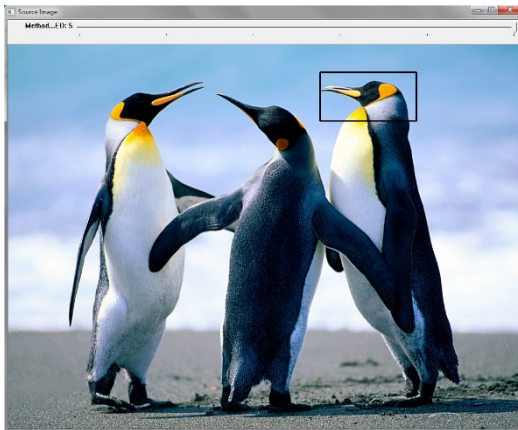


4.2.2 Template Matching: Result 2 (4/4)

- Method: CV_TM_CCOEFF



- Method: CV_TM_CCOEFF_NORMED



Reference

R1. OpenCV Introduction

R1.1 Application Programming Interface (API) Concepts

R2. 2D (Planar) Motions

Content: Topic1. Similarity Measure (相似度量測)

1.0 Computer Vision - Camera Model: Between 3D and 2D

1.1 2D Image

1.2 Camera Model Btw 3D Object and 2D Image

1.3 From Linear Combination to Homography $Ax=b$ (Vector, Matrix)

2.0 Computer Vision (CV) and AI: Similarity Measure = Loss Function

2.1 Sum-of-Squared Differences (SSD) and Mean-Squared Error (MSE)

2.2 Normalized Correlation Coefficient (NCC) and Cross Entropy (CE)

2.3 AI Bayesian Decision Rule - Maximum a Posteriori (MAP) Probability with Gaussian Model

3.0 Image Processing for Deep Learning: Filter, Correlation and Convolution

3.1 Low-Pass Filter - Denoise and High-Pass Filter - Edge Detection

3.2 Correlation Vs. Convolution

3.3 Multi-Resolution - Downsampling (Encoder) and Upsampling (Decoder)

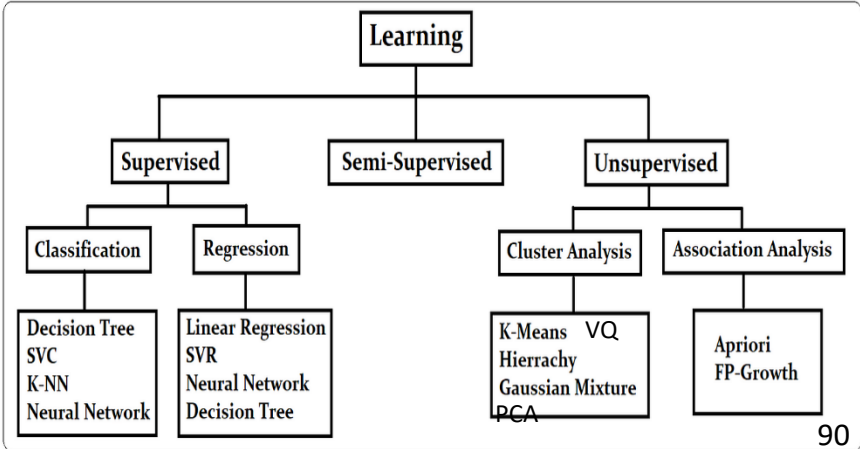
2.3 AI Bayesian Decision Rule - Maximum a Posteriori (MAP) Probability with Gaussian Model



2.3.1 AI Learning: Supervised, Unsupervised and Semi-Supervised

- 1. **Supervised Learning:** Label
 - Is a **problem** with **labeled data** and is a **time-consuming** job, expecting to develop predictive (estimation) capability
 - Is much more **resource-intensive** because of the need for labelled data.
 - Is generally used to classify data or **make predictions**,
 - ➔ **Self-supervised learning:** Makes use of the structure within the data to generate **its own labels**.
- 2. **Unsupervised Learning:** Unlabeled
 - All the observations in the dataset are unlabeled and the algorithms learn to **inherent** structure from the input data
 - Is generally used to **understand relationships within datasets**.
 - Is discovering process, diving into unlabeled data to capture hidden information.
- 3. **Semi-Supervised Learning:** Few are labeled, most are unlabeled
 - **Some** of the observations of the dataset are **labeled** but **most** of them are usually **unlabeled**.
 - So, a **mixture** of supervised and unsupervised methods are usually used.
- 4. **Reinforcement Learning:** Unsupervised (self-learning)
 - Differs from supervised learning in a way that in supervised learning the training data has the answer key with it so the model **is trained with the correct answer itself**
 - whereas in reinforcement learning, there is **no answer** but the reinforcement **agent decides** what to do to perform the given task.

Type	Supervised	Unsupervised	Semi-supervised
Input Data	Labelled	Unlabelled	Partially Labelled
Computational Complexity	Simpler	Computationally Complex	Depends on use of Supervised or unsupervised
Accuracy	Higher	Lesser	Lesser

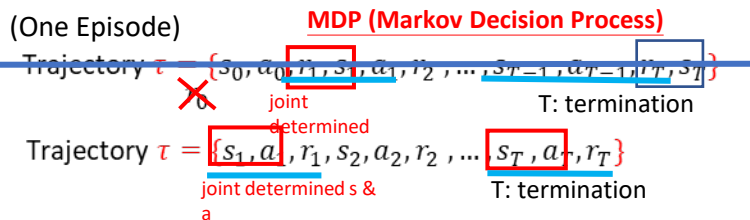


2.3.1 AI Learning: Supervised, Unsupervised and Semi-Supervised

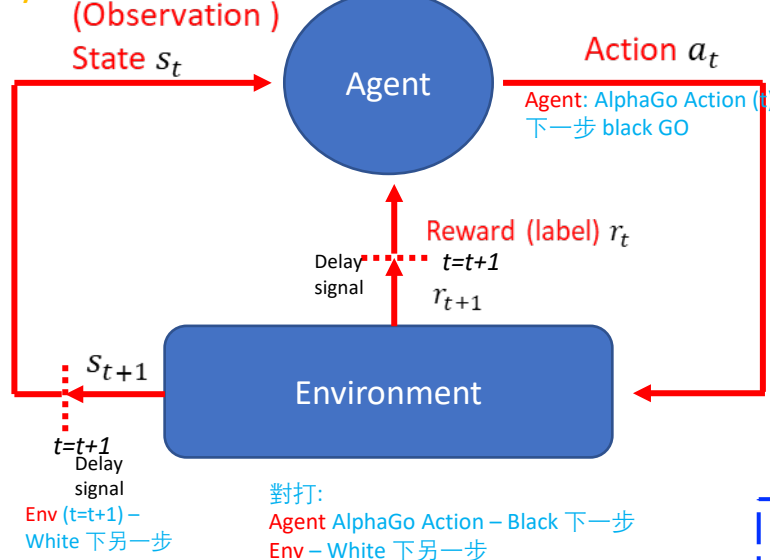
0.1 (1/4): Reinforcement Learning – Scenario detail

P: (from reinforcement learning book:
-State is different from observation
-Example: Go
> **State**: Probably consist of 棋譜 and position of black-and-white 棋子... all information has only one status even under diff. view images
> **Observation**: Different images from different view angles, but having the same (only one) status

Find or define policy for reinforcement learning!
Reinforcement: Markov decision, consider only current t and previous statement t-1
1) **Discrete actions** (video game, GO)
2) **Continuous actions** (robot arm)



-P: Actor follows a policy as a function as the NN. If NN is a deep NN model, then it calls deep reinforcement learning.
-Actor/Policy $\pi(a|s)$: Conditional probability. Given s, take the probably of action a
Actor/Policy $\pi(a_t|s_t)$



Sometimes multiply discount factor γ (0~1)
短視近利 or 重視長遠
Exp. Exponential reduce along the time

This is the reward for only one trajectory:

$$R(\tau) = \sum_{t=1}^T r_t$$

This is average reward over N different trajectories:

$$\bar{R}_\theta = \sum_{\tau} R(\tau) p(\tau|\theta) \approx \frac{1}{N} \sum_{n=1}^N R(\tau^n)$$

Annotations: "Total reward: (long-term reward)", "Expected cumulative reward per episode", "As weight", "over J: Margin??", "J: Similar to HMM: dynamic programming - shortest path".

Agent learns to take actions to maximize expected reward.

$$\theta^* = \arg \max_{\theta} \bar{R}_\theta$$
$$\theta^{new} \leftarrow \theta^{old} + \eta \nabla \bar{R}_{\theta^{old}}$$
$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} (R(\tau^n) - b) \nabla \log p(a_t^n | s_t^n, \theta)$$

Annotations: "An agent (actor) like a function (Neural Network)", "Action = π (Observation) 機率分布: $\pi(a|s)$ ".

Q: How to choose a good actor (policy)?
A: an actor follow the policy which expected reward is maximum than other policy.
That is, the reaction with the environment.

2.3.2 AI Bayesian Decision Rule – (1/3)

Maximum a Posteriori (MAP) Probability with Gaussian Model

1. Maximum a Posteriori (MAP) Probability with Gaussian Model

Decision
1) Posterior Prob. $\propto$ Likelihood Prob. * Priori Prob. Similarity Measure Imbalance, position

2) Likelihood Prob. by Gaussian Model

Prob. Of existing a face? MAP:

$$p(\Omega_f|I) = \frac{p(I|\Omega_f)p(\Omega_f)}{p(I|\Omega_f)p(\Omega_f) + p(I|\Omega_n)p(\Omega_n)}$$

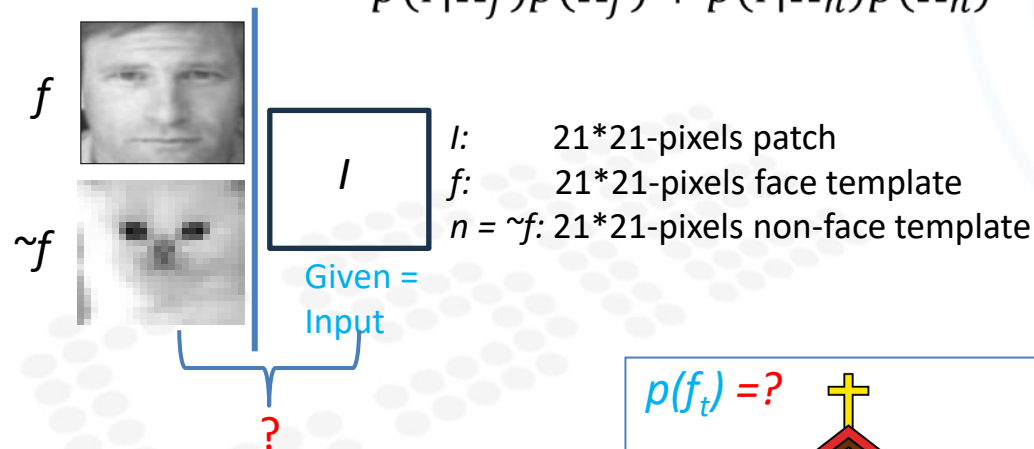
> The similarity btw I and f

$$p(I|f) = \frac{1}{(2\pi)^{1/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(I-f)\Sigma^{-1}(I-f)\right]$$

Covariance matrix $\Sigma = \begin{bmatrix} \sigma_1^2 & 0 & 0 \\ 0 & \sigma_2^2 & 0 \\ 0 & 0 & \sigma_3^2 \end{bmatrix}$

Max:

$$p(\Omega_n|I) = \frac{p(I|\Omega_n)p(\Omega_n)}{p(I|\Omega_f)p(\Omega_f) + p(I|\Omega_n)p(\Omega_n)}$$



4) Loss Function = $p(\Omega_f|I)$

or $p(f_t)$ * Similarity Measure is better
Imbalance

- Similarity Measure:

$$\begin{aligned} \min SSD &= \min(I-f)^2 \\ &= \min(I^2 - 2If + f^2) \\ \min SSD &\rightarrow (I^2 + f^2) - 2(If) \end{aligned}$$

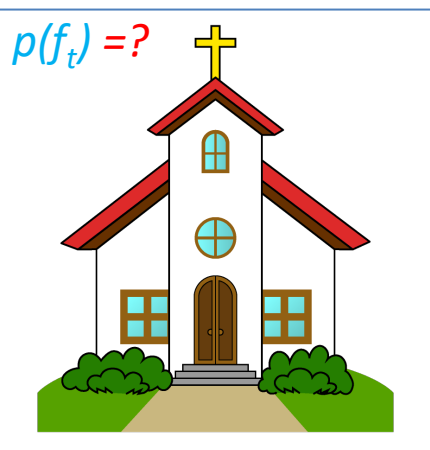
$$\therefore \max CC \rightarrow If$$

1,2,...,4,5,6,...8,9
9,8,...,6,5,4,...2,1

3) Priori Prob. $p(f_t)$ or $p(!f) \neq 1/2$

> Example: position of face appearance probability at time t

$$\begin{aligned} P(x_t) &= P(x_t|x_{t-1})P(x_{t-1}|x_{t-2}) \dots \\ &\cong P(x_t|x_{t-1})P(x_{t-1}) \text{ (causal)} \end{aligned}$$



Deep Learning:

1) MSE

2) Cross Entropy

min

SSD

9-1 = 8

5-5 = 0

max

correlation

9*1 = 9

5*5 = 25

SSD: Sum of Square Difference

CC: Cross Correlation

B1-3 Homography Estimation $Ax=b$

Distance: norm

$r_{Mahalonobis} = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}$

Mahalonobis distance

$d = \sqrt{(m_i - \hat{m}_i)^T (m_i - \hat{m}_i)}$

Euclidean distance = norm 1/2

1. Estimate Homography (H) Between the 3D Model Plane and its 2D Image:

H can be calculated by $\min_H \sum_i \|m_i - \hat{m}_i\|^2$ SSD: Sum of Squared Difference

- Assume 1: m_i is corrupted by Gaussian noise with mean 0 and covariance matrix Λ_{m_i} .

⇒ Probability Density Function (PDF) as likilhood probability:

$P(\cdot) = \frac{1}{(2\pi)^{N/2} |\Lambda_{m_i}|^{1/2}} \exp\left(-\frac{1}{2} (m_i - \hat{m}_i)^T \Lambda_{m_i}^{-1} (m_i - \hat{m}_i)\right)$

N: dimension of m_i , $N = 2$ in our case

⇒ Maximum likelihood estimation of H: $\min_H \sum_i (m_i - \hat{m}_i)^T \Lambda_{m_i}^{-1} (m_i - \hat{m}_i)$

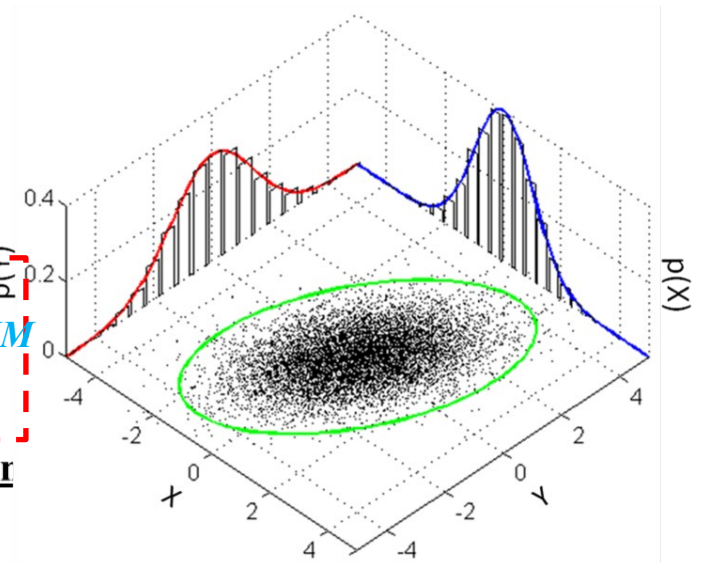
- Assume 2: for all i , $\Lambda_{m_i} = \sigma^2 I$.

$$\Rightarrow \min_H \sum_i (m_i - \hat{m}_i)^T \Lambda_{m_i}^{-1} (m_i - \hat{m}_i) = \min_H \sum_i \|m_i - \hat{m}_i\|^2$$

$m' = HM$

- We can obtain H by applying Levenberg-Marqardt algorithm
 - An initial guess using close form solution
 - Levenberg-Marqardt algorithm

M_i : The 3D model points. (known)
 m_i : The 2D image points. (known)
 $\hat{m}_i$: Expected image points under the homography H.
 $\hat{m}_i = \begin{bmatrix} u \\ v \end{bmatrix} = \frac{1}{\bar{h}_3^T \hat{M}} \begin{bmatrix} \bar{h}_1^T \hat{M} \\ \bar{h}_2^T \hat{M} \end{bmatrix}$



2.3.2 Similarity Measure: Likelihood Probability [0.0, 1.0] (2/3)

2. Bayesian Decision Rule

$$x \in \begin{cases} \Omega_f \\ \Omega_n \end{cases} \begin{cases} \text{if } p(\Omega_f | x) \geq p(\Omega_n | x) \\ \text{otherwise} \\ \text{If } p(f|x) > \text{threshold} \end{cases} \Rightarrow \frac{p(x | \Omega_f)}{p(x | \Omega_n)} \overset{\text{face}}{\underset{\text{nonface}}{\geq <}} \frac{p(\Omega_n)}{p(\Omega_f)} = \gamma \text{ Constant by experiment}$$

1. Likelihood Probability [0.0,1.0]:
 From sum-of-squared differences (SSD) to **Gaussian model (normal distribution) $N(\mu,\sigma^2)$**

$$P(x) = \frac{1}{\sqrt{(2\pi)^D |\Sigma|}} \exp(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu))$$

$$P(I|x) = \frac{1}{\sqrt{(2\pi)^D |\Sigma|}} \exp(-\frac{1}{2} (x - I)^T \Sigma^{-1} (x - I))$$

Covariance Matrix

$$\Sigma = \begin{bmatrix} (x_{11} - \mu)^2 & \cdots & (x_{1D} - \mu)^2 \\ \vdots & \ddots & \vdots \\ (x_{D1} - \mu)^2 & \cdots & (x_{DD} - \mu)^2 \end{bmatrix}$$

➔The similarity btw input unknown patch I and ground truth face x
 I : 24x24-pixels patch; x : Face, $\sim x$: Non-Face

2. Likelihood Probability [0.0,1.0]:
 From cross correlation (CC) to **normalized correlation coefficient (NCC) R**

$$|R(x, y)| = \left| \frac{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(T(x',y') - m^T) * I(x+x',y+y') - m^I]}{\sqrt{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(T(x',y') - m^T)]^2} \sqrt{\sum_{y'=0}^{h'-1} \sum_{x'=0}^{w'-1} [(I(x+x',y+y') - m^I)]^2}} \right|$$

2.3.2 Likelihood Probability by Gaussian Model (3/3-1/2)

1. Gaussian Distribution (Normal Distribution):

1) 當樣本為1-D (univariate)，高斯分佈遵守機率密度函數 (Probability Density Function - PDF):

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

$x \in \{x_1, x_2, \dots, x_n\}$
 $\mu = \frac{1}{n} \sum_{i=1}^n x_n$: mean (平均值或期望值): Average
 $\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$: standard deviation (標準差)

• 右圖為不同平均數與標準差，所對應的機率密度函數曲線。

$N(u, \sigma^2)$
Normal Distribution

• Gaussian Distribution 物理意義:

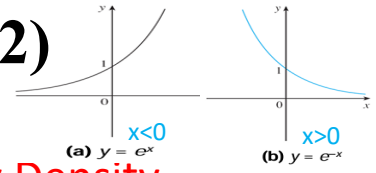
- distance of x from μ
- degree of spread

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

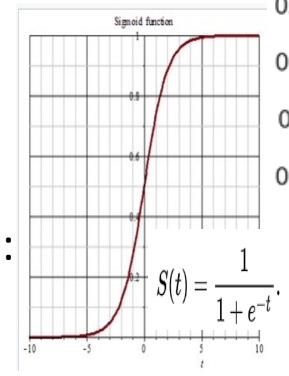
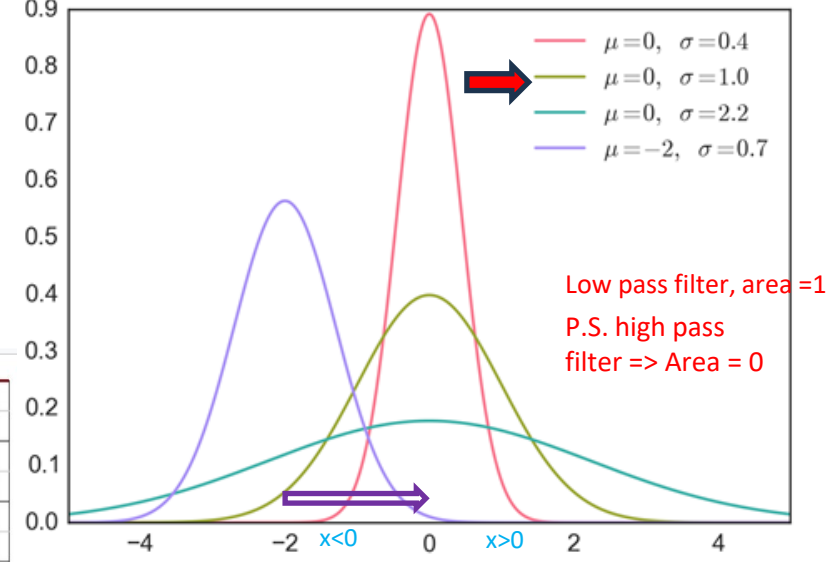
Weight(kg)
10kg
10cm
Height(cm)

- 位移: 回歸基準
- Exponential: 使大小值拉開
- 除標準差: 使 term 統一單位

- SSE $\sum(y-y')^2$
 y : Ground Truth
 y' : Estimation result



不同平均數與標準差，所對應的機率密度函數曲線



- μ : Depend on Center-Position by μ (average),
- σ Depend on height and degree of spread by σ (standard deviation)

- a) Gaussian distribution (Probability)
 $m \pm \sigma$ (68%), $m \pm 2\sigma$ (95%?), $m \pm 3\sigma$ (99%)
- b) Gaussian Mixture Model (GMM)
➔ Weighted GMM estimated by EM

2.3.2 Likelihood Probability by Gaussian Model (3/3-2/2)

2) 當樣本為 n-D (multivariate), x 為 vector (向量化), 高斯分佈:

$$P(x) = \frac{1}{\sqrt{(2\pi)^D |\Sigma|}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

μ : 平均值 ($d \times 1$)

$$\Sigma = \begin{bmatrix} (x_{11} - \mu)^2 & \cdots & (x_{1D} - \mu)^2 \\ \vdots & \ddots & \vdots \\ (x_{D1} - \mu)^2 & \cdots & (x_{DD} - \mu)^2 \end{bmatrix}$$

D: 維度

$|\Sigma|$: 協方差的行列式

協方差矩陣(共變異數矩陣)

- covariance matrix

($d \times d$ symmetric matrix)

Note: 如果沒有 $-\mu$ 就是 correlation

tommy

• Exponential 的物理意義:

$$\exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

• If the variables are independent:

$$P(x) = \prod_i \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left[-\frac{(x_i - \mu_i)^2}{2\sigma_i^2}\right]$$

- SSD $\Sigma(y-y')^2$

y : Ground Truth

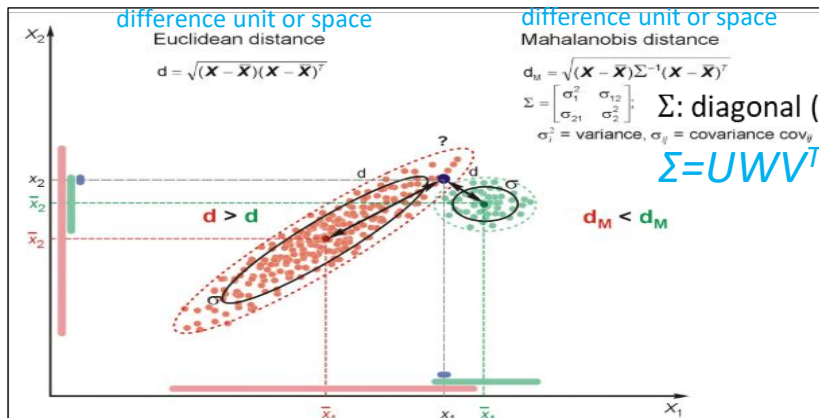
y' : Estimation result

Cannot compare at

difference unit or space

Can compare at

difference unit or space



'Euclidean Distance' VS 'Mahalanobis Distance'

tommy

• Using Gaussian function At the Multivariate and Vector
 Similarity measure $\rightarrow$ 1/Distance Measure

1) SSD / MSD problem $\Rightarrow$ Without Normalization Sigma

2) Mahalanobis Distance $\Rightarrow$ With Normalization Sigma + square root

$$\Sigma = UWV^T$$

• PCA (Principal Component Analysis)

- Gaussian model

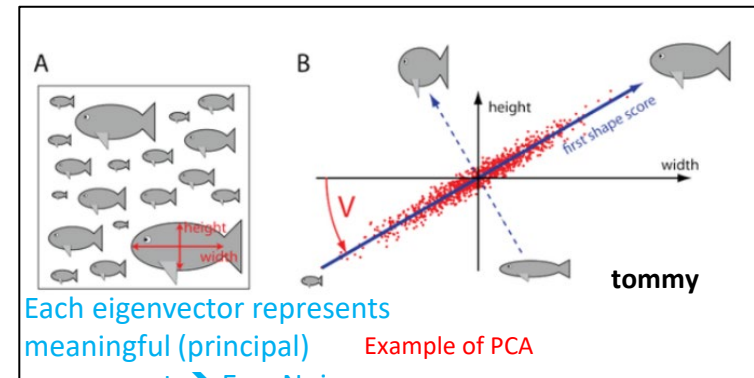
- because it is from Covariance matrix $N(u, \sigma^2)$

1) Covariance matrix ($-u$): Translation

2) Eigenvector U (& V): Direction = Rotation,

3) Eigenvalue W : Variance = Scaling

(present different covariance by PCA)



tommy

1) Gaussian

2) Softmax at DL

2.1) Range of element $\Rightarrow$ (0, 1)

2.2) Sum of elements $\Rightarrow$ 1

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K.$$

Mahalanobias Distance

- Distance of Data Covariance
 - The stronger the correlation between variables, the greater the value.